

FHE for SIMD Arithmetic Logic Units with Amortized $O(1)$ Bootstrapping per Ciphertext

Mingyu Gao^{1,2} and Hongren Zheng¹

¹ Tsinghua University

² Shanghai Qi Zhi Institute

gaomy@tsinghua.edu.cn

zhenghr22@mails.tsinghua.edu.cn

Abstract. The evaluation of both arithmetic and logic operations on machine words (e.g., 64-bit registers) in homomorphic settings is an attractive problem due to its close alignment with real-world programming models. Existing FHE schemes require *iterative* bootstrapping operations with the iteration number scaling with the machine word bit-width n . Prior approaches incur either quadratic cost for multiplication (DM/CGGI), linear cost for logic operations (REFHE [Eurocrypt’26] and Kim25c [eprint/2025/1440]), or logarithmic cost for multiplication (CPL [eprint/2025/1740]).

We mitigate this inherent barrier by amortizing $O(n)$ iterations across $O(n)$ ciphertexts for scenarios with sufficient machine words. We introduce a *triangle encoding* such that in its conversion to boolean mode, each iteration on a combined ciphertext processes multiple ciphertexts, resulting in an *amortized constant* cost per ciphertext. Additionally, the triangle encoding supports leveled arithmetic, and its refreshing requires only two CKKS bootstrapping operations. We reuse discrete CKKS as our boolean mode, which supports lightweight conversion to the triangle encoding. We also introduce a method to induce SIMD structure for the triangle encoding from the RLWE ring.

Even when we bootstrap after each multiplication, we show a $3\times$ to $3.8\times$ higher throughput compared with the state-of-the-art CPL scheme as it requires 3 to 5 bootstrapping operations (approximately $\log(n/4)$ with n ranging from 64 to 256). We further estimate $15\times$ and $491\times$ higher throughput for multiplication and bitwise operations, respectively, compared with the REFHE scheme. We make our OpenFHE-based prototype open source for ease of reference.

1 Introduction

Fully Homomorphic Encryption (FHE) enables unbounded computation on encrypted data. It was a long-standing open problem [RAD78] until Gentry made a breakthrough [Gen09] by introducing the bootstrapping technique. Subsequent early efforts like [vdGHV10, SV10, BV11, GHS12, GSW13] have made various improvements to make FHE more practical. Most of them are based on modern hardness problems like LWE [Reg05] and its variants like Ring-LWE and Polynomial-LWE [LPR10, LPR13, SSTX09].

Modern schemes often excel at different types of operations. For arithmetic-intensive tasks, there are various choices, including exact schemes such as BGV [BGV12], BFV [Bra12, FV12], CLPX [CLPX18], GBFV [GV25, CHM+25], and dBFV [PZZ25], as well as approximate schemes like CKKS [CKKS17] and later high-precision adaptations like [LLL+21, CCKS23]. Boolean operations, in contrast, are typically supported through scheme-switching with substantial computation overhead [BGGJ20, LHH+21, LMP22, BCK+23].

In this context, a recent trend has emerged towards natively supporting both arithmetic and boolean operations on machine words (e.g., `int64`). This design point is particularly appealing because it closely resembles the hardware arithmetic logic unit (ALU) and aligns with real-world programming models. We broadly categorize these approaches into two groups; a concise summary is provided in Table 1.

Scalar Arithmetic and Boolean Schemes. This category supports arithmetic and boolean operations upon a single scalar machine word. Schemes such as DM [DM15] (with variants like LMKCDEY [LMK+23]) and CGGI [CGGI20] (also known as TFHE) natively support boolean operations by decomposing the scalar into bits (or digits) and encrypting one bit (or digit) per ciphertext. However, arithmetic operations like addition and multiplication are emulated through circuits. In their textbook multiplication circuit, the number of bootstrapping operations required is *quadratic* in the bit-width [BdBB+25].

More recently, REFHE [BFG+25] adopts the polynomial plaintext modulus from CLPX/GBFV in a BGV style. It uses a non-cyclotomic polynomial to directly induce the machine-word algebra inside one ciphertext, with the decomposed bits living in distinct polynomial coefficients. They support leveled arithmetic operations, while boolean operations are possible during *iterative* bootstrapping using the method from [KDE+24], with the number of bootstrapping required being *linear* in the bit-width.

These scalar schemes align perfectly with CPU-like computation models and are flexible and low-footprint for small amounts of data. However, their scalability to data-intensive workloads (e.g., machine learning, databases, analytics, and GPU-style vector processing) can be troublesome because they cannot exploit the amortization benefits of SIMD execution.

SIMD Arithmetic and Boolean Schemes. A variant of CKKS called *discrete* CKKS [DMPS24, BCKS24, BKSS24, AKP25] was recently proposed to support discrete data units (bits or small integers) and look-up table (LUT) over them through bootstrapping. Building upon this, later works advance to high-precision arithmetic. [BK25] focuses on arithmetic over arbitrary plaintext modulus using homomorphic residue number system, while not supporting boolean operations due to lack of exact reduction. For power-of-two machine words with bit-width n (up to 2048 bits), [Kim25b, Kim25c, CPL25] decompose them into k digits with $k = n/4$, and simulate carry propagation for each digit through bootstrapping, therefore supporting both arithmetic and boolean operations.

In more detail, [Kim25c] requires $O(\log k)$ CKKS bootstrapping for reducing the multiplication result to a redundant representation, while requiring $O(k)$

Table 1. Comparison of arithmetic and boolean FHE schemes.

Scheme	Type [†]	Arith [‡]	Bool	A→B	B→A
DM/CGGI	Scalar	$O(n)$ or $O(n^2)$	$O(n)$	N/A	N/A
REFHE	Scalar	Leveled w/ $O(n)$	$O(n)$	$O(n)$	Repack
[Kim25c]	SIMD	$O(\log k)$	Leveled w/ $O(1)$	$O(k)$	Free
[CPL25]	SIMD	$O(\log k)$	Leveled w/ $O(1)$	$O(1)$	Free
Our Scheme	SIMD	Leveled w/ $O(1)$	Leveled w/ $O(1)$	Amort. $O(1)$	Lightweight

[†] Scalar schemes count the number of gate bootstrapping (PBS); SIMD schemes count the number of CKKS bootstrapping.

[‡] “Leveled” stands for allowing leveled execution, and “w/” stands for the number of bootstrapping operations required for refreshing noise. Otherwise the notation is the required number of bootstrapping operations after *each* operation. k is the number of digits and usually $k = n/4$, with n being the bit-width of machine words.

CKKS bootstrapping to reduce to an exact representation; the state-of-the-art [CPL25] improves the construction to requiring $O(\log k)$ CKKS bootstrapping for exact representation by introducing lightweight carry propagation. They require the reduction process after *each multiplication* for carry management and cleaning the higher bits. To support boolean operations like bitwise logic, they require further bootstrapping to decompose digits into bits.

Focusing on the SIMD regime for data-intensive workloads, we conclude that the current constructions are *heavyweight* and *bootstrapping-intensive*. This raises the following question:

*Is there a SIMD arithmetic and boolean FHE scheme
with less bootstrapping?*

in the sense that *leveled* operations are allowed before refreshing/conversion, and the number of CKKS bootstrapping per ciphertext can be amortized to *constant* if there is a sufficient amount of data (inherent reasons for amortization are explained in the technical overview section).

1.1 Our Contributions

We give an affirmative answer by constructing a CKKS-style FHE scheme with arithmetic and boolean modes, such that, from the cryptographical perspective,

1. Enables *leveled* operations, either all in arithmetic or all in boolean, before a need for refreshing.
2. Takes a *constant* number of CKKS bootstrapping operations for refreshing inside each mode.
3. Supports *lightweight* conversion from boolean to arithmetic mode when the bit-width n is moderate.

4. Takes an *amortized constant* number of CKKS bootstrapping operations *per ciphertext* for conversion from arithmetic to boolean mode, by using $n/4$ CKKS bootstrapping operations to process $n/4$ ciphertexts.

From the application perspective, our scheme

1. Supports both *arithmetic and boolean* (e.g., bitwise logic, bit shift/rotation and comparison) operations, providing a programming interface most developers are familiar with.
2. Supports *machine-word integers* (e.g., 8-bit, 64-bit and 256-bit registers), matching the bit-width of commodity hardware.
3. Supports *SIMD*, matching the execution model of GPU and vector processors for data-intensive workloads.

We envision that our scheme is suitable for *massive* data processing (e.g., more than 16384 machine words) with the goal of *high-throughput* for workloads that have arithmetic and boolean operations on *moderate* bit-width (e.g., 64-bit) data. The comparison of our scheme with previous schemes can be checked in Table 1.

Our main technical novelties consist in

1. *Triangle encoding* that is a balance between arithmetic and boolean representations, enabling leveled arithmetic operations and easy management.
2. *Amortized constant* cost for *inherently* iterative arithmetic-to-boolean conversion, leveraging structural properties of the RLWE problem.
3. *SIMD structure* for the triangle encoding through grouping complex slots.

They are detailed in the technical overview section.

We emphasize that our scheme is an “exact FHE scheme” and achieves *exact correctness* (negligible failing probability) under application-aware security model (See Sec. 7.1 for more discussion on security notions) instead of *approximate correctness* even though we are CKKS-compatible, as we represent data in discrete form and can clearly separate noise from messages (Theorem 1).

We present concrete *performance numbers* in Table 2. Even when we bootstrap after each operation, we achieve the least bootstrapping number (2 bootstrapping for arithmetic and 1 bootstrapping for bitwise boolean), and concretely outperform [Kim25c] for boolean operations and outperform [CPL25] for multiplication in experiments. Compared with scalar schemes like TFHE-rs and REFHE, we achieve significantly higher throughput. It is worth noting that our 256-bit multiplication achieves a $7\times$ lower latency than that of TFHE-rs.

We also provide an OpenFHE-based *open source* prototype³ for ease of reference and adoption.

1.2 Technical Overview

We have three design goals: (1) enabling both arithmetic and boolean operations, (2) amortized constant number of bootstrapping per ciphertext, and (3) SIMD.

³ <https://github.com/tsinghua-ideal/fhe-simd-alu>

Table 2. Performance numbers of our scheme even when we bootstrap after each operation (A2A for multiplication, either B2B or B-A2B[‡] for bitwise) compared with other schemes. Experiments were conducted in a single-threaded setting on the same machine for our scheme and TFHE-rs (CPU backend). All parameters satisfy 128-bit security. “Latency” (Lat.) is the execution time of all bootstrapping operations, and “Amortized time” (Amort.) is obtained by dividing latency by the number of slots.

Scheme	Dim	BitW	#Slots	Mult (Lat./Amort.)	Bitwise (Lat./Amort.)
TFHE-rs	2^{11}	64	1	11.4s	209.9ms
		256	1	188.9s	876.6ms
REFHE	2^{12}	64	1	200ms	$\approx 3.83s^*$
[Kim25c] [†]	2^{16}	64	2048	24.9s / 12.2ms (4 BTS)	105s / 51.8ms (17 BTS)
		256	512	31.9s / 62.3ms (5 BTS)	414s / 0.81s (65 BTS)
[CPL25] [†]	2^{16}	64	1024	40.3s / 39.3ms (3 BTS)	14.3s / 13.1ms (1 BTS)
		256	256	74.8s / 292ms (5 BTS)	14.9s / 58.4ms (1 BTS)
Our Scheme	2^{16}	64	1024	13.5s / 13.2ms (2 BTS)	6s / 6ms (1 BTS)
					128s / 7.8ms (1 BTS) [‡]
		256	256	19.5s / 76.0ms (2 BTS)	6s / 23ms (1 BTS)
					823s / 50ms (1 BTS) [‡]

* Rough estimation from REFHE: “Complexity of 64-bit multiplication using TFHE is twice(thrice) the cost of bootstrapping in REFHE”.

[†] The experimental data is borrowed from their paper under different codebase and environment. The data for bitwise operation is estimated based on multiplication.

[‡] Batched A2B processes 16 (or 64) ciphertexts for $n = 64$ (or $n = 256$).

We first explain the drastic difference between two kinds of operations.

Two Extremes of the Spectrum. Arithmetic representations (friendly for multiplication) and boolean representations (for bitwise logic) are at two extremes of the plaintext algebra spectrum. The former prefers the number to be represented as a “whole” (either in coefficient domain or in frequency domain), while the latter prefers the number to be represented as “bit/digit vector”. A conversion between them is inherently *iterative*, which should be able to either (1) assemble the bit/digit vector into a whole, or (2) decompose the whole into a bit/digit vector. Informally, the decomposition is harder than the assembly. Therefore “bit/digit vector” is the primary choice for existing FHE schemes that target both kinds of operations, as it is easier to decompose prior to encryption and simulate the “whole” arithmetic during homomorphic computation.

However, the choice of “bit/digit vector” will inevitably lead to the management of three objects during homomorphic arithmetic: (1) *carries*, in that the computation results of each bit/digit will affect other bits/digits, (2) *arithmetic overflows*, in that multiplying two 64-bit integers will produce a 128-bit integer but the higher 64-bit is often required to be discarded, and (3) *noise*, which is inherent to the LWE problem.

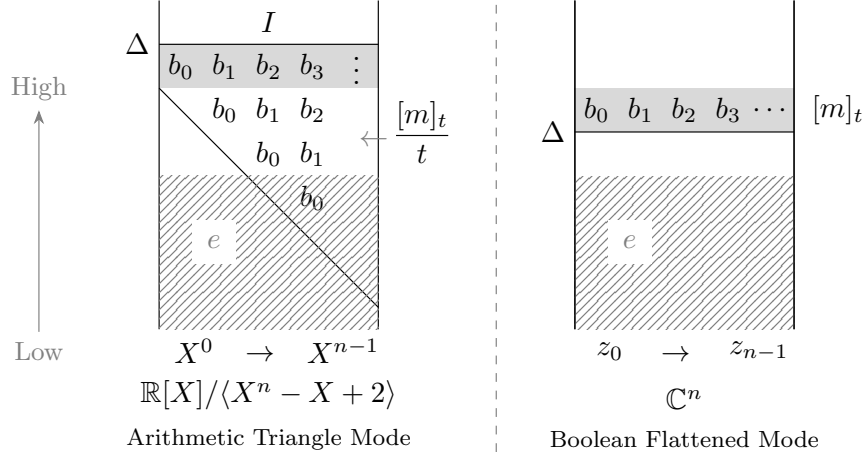


Fig. 1. (Left) The triangle shape of arithmetic encoding from Def. 10 with the $\frac{m}{2^n}$ term not shown. The upper side (gray region) contains essential bits to recover m . The noise e (shadow region) can contaminate part of the triangle. (Right) The flattened shape of boolean encoding, with each bit in each complex slot.

Existing schemes need to handle some of them *iteratively*, which appears to be unavoidable in the “bit/digit vector” paradigm. DM/CGGI, [Kim25c], and [CPL25] need to iteratively simulate carry propagation and leave spaces for arithmetic overflows. REFHE [BFG⁺25] uses algebraic constraints to propagate carries on the bit vector *at once*, but the noise and arithmetic overflows are closely coupled with the bit vector and require iterative refreshing process. By “at once”, we mean a constant number of regular or bootstrapping operations.

Therefore, we drift away from the existing “bit/digit vector” paradigm by introducing the “triangle” encoding (Fig. 1) to mitigate the *iterative* nature using amortization. The encoding could be viewed as a middle ground of the “whole” paradigm and the “bit vector” paradigm. For a message $m \in \mathbb{Z}_{2^n}$, let $[m]_t = \sum_{i=0}^{n-1} b_i X^i$ be the *flattened* polynomial with binary coefficients where b_i is the i -th bit of m , we multiply the flattened polynomial by t^{-1} where $t = X - 2$ in an auxiliary ring to form a triangle: the i -th coefficient contains the lower $i + 1$ bits of m . Note that the $(n - 1)$ -th coefficient is the “whole”, and the upper side of the triangle is the “bit vector”. If we consider other terms during computation, the general form of the triangle encoding is

$$\Delta I + \frac{\Delta}{X - 2} [m]_t + e \quad \text{in } \mathbb{R}[X]/\langle X^n - X + 2 \rangle$$

where Δ is a CKKS-style scaling factor, arithmetic overflow I is a polynomial with integer coefficients, and e is the noise polynomial.

Thanks to the division-by- t and the multiplication of the scaling factor Δ , the arithmetic overflows and noise can be refreshed *at once* as they are represented in a coefficient-wise manner. We also note that, this form can be maintained across

addition/multiplication, so this encoding allows *leveled* execution of arithmetic operations until a need for managing the overflow I and/or the noise e .

It is worth noting that while the noise e can contaminate part of the triangle, as long as its upper side is kept clean, we can clearly separate e from the message by a multiplication of $X - 2$ and we obtain *exact correctness*.

Now we discuss the boolean mode and the conversions. We first define the encoding in boolean mode to be “bit vector” (Fig. 1):

$$\Delta(b_0, b_1, \dots, b_{n-1}) + e \quad \text{in } \mathbb{C}^n$$

which is exactly discrete CKKS for bits. Bitwise logic and rotation can be naturally implemented as *leveled* CKKS addition/multiplication and rotation. The refreshing inside boolean mode is exactly one discrete CKKS bootstrapping. Note that the “bit vector” is exactly $[m]_t$, therefore a linear transformation and scalar multiplication gives us the “triangle” encoding, which is *lightweight* when the bit-width n is moderate.

Now we face the challenging conversion from the “triangle” mode to the “bit vector” mode. We admit that the conversion is still *iterative* with the presence of the arithmetic overflow I ; indeed, such iterative nature appears to be an inherent barrier, as an efficient extraction of bits is equivalent to an efficient homomorphic modular reduction operation, which has been mitigated in various ways in previous literature (e.g., sine approximation). Instead, we crucially observe that it is possible to combine the “triangle” shape with the structural “limitation” of the underlying RLWE hardness problem to *amortize* the cost of such iterative process, which is detailed below.

Amortized Constant at Ciphertext Level. By “limitation” we are referring to the practical parameters of RLWE [ACC+18, BCC+24]. Its ring dimension N ranges from 2^{10} to 2^{17} , while machine words are often small like 64-bit. Therefore, the basic operational unit is a *ciphertext* and is substantially large.

One crucial way to amortize such overhead is through SIMD. We will discuss the *regular* SIMD packing later. Here we discuss a *special* SIMD during arithmetic-to-boolean conversion.

Our *novel* observation is that, each iteration only needs *partial* information from one machine word, therefore *multiple* ciphertexts can be *combined* into one ciphertext for one iteration, and the cost of multiple iterations can be amortized to become *constant*.

In more detail, we only process one “corner” of the triangle for each machine word in each iteration. After one iteration, we remove the corner (and corresponding bits in some other coefficients) from the triangle, resulting in a *smaller* triangle, so such iterative process can continue without complexity blowup.

SIMD Homomorphism Chain. Now we discuss how to enable regular SIMD packing. Previously, SIMD structure is closely coupled with the RLWE ring. As pronounced in schemes like BGV, BFV, CLPX, and GBFV, the available plaintext algebra is directly related to the choice of the RLWE ring. For example, if we employ the implementation-friendly RLWE ring $\mathbb{Z}[X]/\langle X^N + 1, Q \rangle$ with power-of-two cyclotomic polynomial, the available plaintext algebra $\mathbb{Z}_p$ is often limited to cases where p is a prime of special form.

To enable the machine-word algebra, REFHE deviates away from such regime by using a non-cyclotomic arithmetic ring $\mathbb{Z}[X]/\langle X^n - X + 2 \rangle$. However, the problem of inducing SIMD structure *within* this ring remains *open*, and we do not intend to answer it here.

Instead, our key observation is to induce SIMD structure through complex slots. Such connection decouples the RLWE ring and the target arithmetic ring. In principle, any ring with a connection to the complex slots can be utilized, as long as the transformation does not “blow up the noise”. For convenience and concreteness, we directly use the power-of-two RLWE ring as follows.

First, CKKS connects the power-of-two RLWE ring with $N/2$ numbers of complex slots:

$$\mathbb{Z}[X]/\langle X^N + 1, Q \rangle \quad \leftarrow \quad \mathbb{R}[X]/\langle X^N + 1 \rangle \quad \cong \quad \mathbb{C}^{N/2}$$

The “ $\leftarrow$ ” stands for rounding a polynomial with real coefficients into integer coefficients. We then note a homomorphism $\mathbb{C}^{N/2} = (\mathbb{C}^{n/2})^{N/n}$ from NeuJeans and Homomorphic Integer Computer [JPK⁺24, Kim25c] that *groups* the complex slots. We therefore connect each machine-word algebra to each group of complex slots, with new connections highlighted in blue:

$$\begin{array}{ccc} \mathbb{Z}[X]/\langle X^N + 1, Q \rangle & & \\ \uparrow & & \\ \mathbb{R}[X]/\langle X^N + 1 \rangle & \cong \mathbb{C}^{N/2} = \left(\mathbb{C}^{n/2} \right)^{N/n} \cong & (\mathbb{R}[X]/\langle X^n - X + 2 \rangle)^{N/n} \\ & & \uparrow \Delta \\ & \mathbb{Z}_{2^n}^{N/n} \cong & (\mathbb{Z}[X]/\langle X^n - X + 2, X - 2 \rangle)^{N/n} \end{array}$$

Fig. 2. The SIMD Homomorphism Chain

As indicated in the tail of the homomorphism chain, we get SIMD slots of machine-word algebra from one RLWE ring.

1.3 Sketch from Application Perspective

This section intends to give a sketch of the execution flow, parameters, and practical implications of our scheme for concrete understanding.

Computation. First, concrete computation is conducted in the RLWE ring $\mathbb{Z}[X]/\langle X^N + 1, Q \rangle$. All our triangle constructions, rings, and homomorphism chain can be understood as *encoding*, that happens prior to the encryption. All homomorphic operations are built upon prior CKKS operations and existing hardware/software acceleration efforts for CKKS can be used for this scheme. Section 6 gives the whole picture of operations needed and offered by our scheme.

Execution Flow. We have two major user-facing ciphertext formats: arithmetic mode and boolean mode, decided at encoding time. Ciphertext in each mode supports *leveled* execution until we are required to *refresh* it, or *convert* it to another mode.

Packing. There are full packing and sparse packing in our scheme, in the same sense as in the CKKS literature. We mainly discuss full packing in this work, but similar algebra applies for sparse packing (Appendix A.1). For full packing, while one ciphertext in arithmetic mode can pack N/n machine words, the boolean mode can only pack $N/(2n)$ machine words, so arithmetic-to-boolean conversion for one ciphertext will produce two ciphertexts, each containing lower and upper halves of the machine words (Fig. 5). These boolean-mode ciphertexts can be combined into one CKKS ciphertext with the upper halves in the imaginary part for bootstrapping.

“Leveled”. Unlike previous schemes where the number of available levels is more related to the number of RNS moduli, the number of levels in our scheme depends on the noise. Roughly speaking, for the scaling factor $\log \Delta \approx 43$ in our example parameter, we allow roughly 33 bits of noise for bootstrapping to function. We start from the bottom noise of roughly 20 bits, and each level of multiplication will incur 2 to 6 bits of noise growth. Therefore, the example parameter supports a multiplication circuit of depth 2 to 3 for full multiplication (Def. 13), or 4 to 5 for short multiplication (e.g., plaintexts, fresh ciphertexts, ciphertexts converted from boolean mode). We comment that selecting a larger Δ during parameter generation could allow deeper multiplication. Dynamic scaling factor technique [CCK⁺25] could also enable deeper multiplication.

For CKKS level consumption, the full multiplication in our scheme takes *two* CKKS levels, though it can be fused with other multiplication to reduce level consumption. The short multiplication takes one CKKS level.

As the behavior of “leveled” is complex in our scheme, we consider software control/compiler for this scheme to be helpful future work (also for security concerns, see Sec. 7.1).

2 Preliminaries

Notations. We use normal font m for an element in a ring like $m \in \mathcal{R}_Q$, $m \in \mathcal{Z}_t$, $m \in \mathbb{Z}_{2^n}$ and $m \in \mathbb{C}$. We will state clearly which ring it is in. When the ring is a polynomial ring, we may also write it as $m(X)$ when we want to emphasize the role of X . For taking the i -th coefficient of the polynomial, we use the notation m_i and define $m(X) = \sum_{i=0}^{N-1} m_i X^i$.

For a vector of elements, we use bold font like $\mathbf{m}$. One example is $\mathbf{m} = (m_0, m_1, \dots, m_{N-1}) \in \mathbb{C}^N$. Vectors are column vectors. We may also sometimes identify a polynomial $m(X)$ as a vector $\mathbf{m}$. We use bold capital letter for matrices like $\mathbf{A} \in \mathbb{C}^{M \times N}$, and let $A_{i,j}$ be the element in the i -th row and j -th column, for $0 \leq i < M$ and $0 \leq j < N$. We use $(\cdot; \cdot)$ as vertical concatenation of two matrices, namely $(\mathbf{A}; \mathbf{B}) = \begin{bmatrix} \mathbf{A} \\ \mathbf{B} \end{bmatrix}$.

We use infinity norm $\|\cdot\|_\infty$ for the norm operator $\|\cdot\|$, either on polynomials, vectors, or matrices. We use base 2 for taking logarithm $\log(\cdot)$.

2.1 Rings and their relations

Here we introduce all essential ring notations used in this work. More relations and explicit expressions can be checked in Appendix A.1.

RLWE Ring $\mathcal{R}_Q$. We choose a power-of-two number N . It is the ring dimension for the RLWE ring $\mathcal{R} = \mathbb{Z}[X]/\langle X^N + 1 \rangle$. Q is a number related to the RLWE problem. In usual deployment, Q can be decomposed into smaller primes like $Q = Q_L = q_0 q_1 \cdots q_L$. We use the notation $\mathcal{R}_Q = \mathcal{R}/Q\mathcal{R} = \mathbb{Z}[X]/\langle X^N + 1, Q \rangle$ and $\mathcal{R}_{\mathbb{R}} = \mathbb{R}[X]/\langle X^N + 1 \rangle$. For concreteness, one can think of $N = 2^{16}$ and $Q \approx 2^{1200}$.

Arithmetic Ring $\mathcal{Z}_{\mathbb{R}}$. We choose a power-of-two number n . It is the ring dimension for the ring $\mathcal{Z} = \mathbb{Z}[X]/\langle X^n - X + 2 \rangle$. We denote the polynomial modulus $t = X - 2$ in $\mathcal{Z}$. We use the notation $\mathcal{Z}_t = \mathbb{Z}[X]/\langle X^n - X + 2, X - 2 \rangle$ and $\mathcal{Z}_{\mathbb{R}} = \mathbb{R}[X]/\langle X^n - X + 2 \rangle$. We call $\mathcal{Z}_{\mathbb{R}}$ the arithmetic ring and it is the primary ring we are working on. For concreteness, one can think of $n = 64$.

Connection to $\mathbb{C}$. For the RLWE ring $\mathcal{R}_{\mathbb{R}}$, $X^N + 1$ is the $(2N)$ -th cyclotomic polynomial. Let $\zeta = \exp(2\pi\sqrt{-1}/(2N))$, CKKS chooses $N/2$ primitive roots of unity as $\{\zeta_j : 0 \leq j < N/2\}$ where $\zeta_j = \zeta^{5^j}$. Note that they together with their conjugates $\{\bar{\zeta}_j : 0 \leq j < N/2\}$ form all the roots for $X^N + 1$ in $\mathbb{C}$.

Therefore, we define the *canonical embedding* $\sigma : \mathcal{R}_{\mathbb{R}} \rightarrow \mathbb{C}^{N/2}$ that $\sigma = (\sigma_0, \sigma_1, \dots, \sigma_{N/2-1})$ where $\sigma_j : \mathcal{R}_{\mathbb{R}} \rightarrow \mathbb{C}$ is defined as

$$\sigma_j(m(X)) = m(\zeta_j) \quad , 0 \leq j < N/2$$

Each σ_j is a ring homomorphism and σ is a ring isomorphism.

For the arithmetic ring $\mathcal{Z}_{\mathbb{R}}$, $X^n - X + 2$ has n complex roots for $n \geq 2$ and n being a power-of-two. We pick $\{\xi_j : 0 \leq j < n/2\}$ such that ξ_j are in the upper half of the complex plane. We make an arbitrary choice to sort them using their real part, with the largest (closest to 1) being ξ_0 . Similarly, we can define the *$\mathcal{Z}$ -canonical embedding* $\tau : \mathcal{Z}_{\mathbb{R}} \rightarrow \mathbb{C}^{n/2}$ that $\tau = (\tau_0, \tau_1, \dots, \tau_{n/2-1})$ where $\tau_j : \mathcal{Z}_{\mathbb{R}} \rightarrow \mathbb{C}$ is defined as

$$\tau_j(m(X)) = m(\xi_j) \quad , 0 \leq j < n/2$$

Note that τ is a ring isomorphism.

Isomorphisms in matrix form. It will be convenient to express the isomorphisms $\sigma : \mathcal{R}_{\mathbb{R}} \rightarrow \mathbb{C}^{N/2}$ and $\tau : \mathcal{Z}_{\mathbb{R}} \rightarrow \mathbb{C}^{n/2}$ as linear transformations in matrix form so we can find its inverse and evaluate them *homomorphically*.

In the RLWE ring $\mathcal{R}_{\mathbb{R}}$, fix a polynomial $m(X)$ and identify it as a vector $\mathbf{m} = (m_0, m_1, \dots, m_{N-1}) \in \mathbb{R}^N$, σ can be understood as an $(N/2) \times N$ Vandermonde matrix $\mathbf{U}_\sigma$ such that $\mathbf{z} = \mathbf{U}_\sigma \mathbf{m}$ with $\mathbf{z} \in \mathbb{C}^{N/2}$. Consider the $N \times N$ matrix $\text{CRT}_\sigma = (\mathbf{U}_\sigma; \overline{\mathbf{U}_\sigma})$, its inverse is $\text{CRT}_\sigma^{-1} = \frac{1}{N} \overline{\text{CRT}_\sigma}^T$. We have $(\mathbf{z}; \bar{\mathbf{z}}) = \text{CRT}_\sigma \cdot \mathbf{m}$, therefore the inverse transform can be evaluated using $\frac{1}{N} \overline{\mathbf{U}_\sigma}^T$. We can further

divide $\mathbf{U}_\sigma$ into two $(N/2) \times (N/2)$ square matrices $\mathbf{U}_\sigma = [\mathbf{U}_{\sigma,0} \quad \mathbf{U}_{\sigma,1}]$. Note that $\mathbf{U}_{\sigma,1} = \sqrt{-1} \cdot \mathbf{U}_{\sigma,0}$, therefore we can use $\mathbf{U}_{\sigma,0}$ for evaluation. We use the notation $\mathbf{V}_{\sigma,0} = \overline{\mathbf{U}_{\sigma,0}}^T$.

In the arithmetic ring $\mathbb{Z}_\mathbb{R}$, fix a polynomial $m(X)$ and identify it as a vector $\mathbf{m} = (m_0, m_1, \dots, m_{n-1})$. τ can be understood as an $(n/2) \times n$ Vandermonde matrix $\mathbf{U}_\tau$ such that $\mathbf{z} = \mathbf{U}_\tau \mathbf{m}$ with $\mathbf{z} \in \mathbb{C}^{n/2}$.

For $\text{CRT}_\tau = (\mathbf{U}_\tau; \overline{\mathbf{U}_\tau})$, we denote $\tau^{-1} = \text{CRT}_\tau^{-1}$. Vandermonde matrix theory tells us that the k -th column of the inverse τ^{-1} has the coefficients of the k -th Lagrange polynomial. Let $\ell_k(X) = \sum_{i=0}^{n-1} \ell_{k,i} \cdot X^i$, then τ^{-1} can be expressed as $\tau^{-1} = [\mathbf{V}_\tau \quad \overline{\mathbf{V}_\tau}]$ where $\mathbf{V}_\tau$ is a $n \times (n/2)$ matrix. We further divide them into $(n/2) \times (n/2)$ square matrices $\mathbf{U}_\tau = [\mathbf{U}_{\tau,0} \quad \mathbf{U}_{\tau,1}]$ and $\mathbf{V}_\tau = (\mathbf{V}_{\tau,0}; \mathbf{V}_{\tau,1})$.

2.2 The RLWE problem and its operations

We state a definition of the RLWE problem used in production for exposition.

Definition 1 (The RLWE problem). *Let N be a power-of-two number and Q be a number, χ_s and χ_e be the secret key and noise distribution. In the ring $\mathcal{R}_Q = \mathbb{Z}[X]/\langle X^N + 1, Q \rangle$, we sample a uniformly random $a \in \mathcal{R}_Q$, a secret key s from χ_s and a noise e from χ_e , let $b = -a \cdot s + e$, we return (b, a) .*

Search RLWE problem consists in finding such secret key s from (b, a) . Decision RLWE problem consists in distinguishing between (b, a) from a uniformly random sample $(b', a') \in \mathcal{R}_Q^2$.

With the RLWE problem, we are able to encrypt a message $m \in \mathcal{R}_Q$. We state the secret-key encryption for exposition. It can be easily adapted to the public-key version (e.g., [KPZ21]).

Definition 2 (RLWE Encryption/Decryption). *Given $m \in \mathcal{R}_Q$ and a secret key s , we can sample a uniformly random $a \in \mathcal{R}_Q$ and a noise e from χ_e and produce a ciphertext*

$$\begin{aligned} \text{ct} &= \text{RLWE.Encrypt}_s(m) = (-a \cdot s + m + e, a) \in \mathcal{R}_Q^2 \\ \text{RLWE.Decrypt}_s(\text{ct}) &= \langle \text{ct}, (1, s) \rangle = m + e + QI \equiv m + e \pmod{Q} \end{aligned}$$

where I is a polynomial with integer coefficients. The exact decoding procedure is scheme-specific.

We now highlight the shape of I .

Proposition 1 (Distribution of the overflow I in RLWE, [BTH22]). *The I in the decryption formula follows Irwin-Hall distribution parameterized by the secret key distribution χ_s .*

For sparse ternary secret key of Hamming weight 32, we have $\|I\|_\infty < 16$ with negligible failing probability.

We remark that the secret key of Hamming weight 32 is not our main secret key; it is the *encapsulated* key only for ModRaise as defined below.

While in normal cases the overflow is not present in the decrypted plaintext, it is present in the high bits after a modulus raising operation that regains level.

Definition 3 (RLWE Modulus Raising). *Given $\text{ct} = (b, a) = (-a \cdot s + m + e, a) \in \mathcal{R}_{Q_{L-1}}^2$ with $m \in \mathcal{R}_{Q_{L-1}}$. Suppose $Q_L = Q_{L-1}q_L$, we can raise from $\mathcal{R}_{Q_{L-1}}$ to $\mathcal{R}_{Q_L}$ by the following operation*

$$\text{ct}' = \text{RLWE.ModRaise}_{q_L}(\text{ct}) = (b, a) \in \mathcal{R}_{Q_L}^2$$

which is directly putting elements from the smaller ring to the larger ring.

The effect on the message is

$$\text{RLWE.Decrypt}_s(\text{ct}') \equiv m + e + IQ_{L-1} \pmod{Q_L}$$

when $\|I\|_\infty \leq q_L$. By using the encapsulated sparse key with small Hamming weight, the $\|I\|$ after ModRaise is less than 16, and this is used to achieve negligible failing probability for CKKS bootstrapping [BTH22].

Conversely, it is possible to use the modulo-by- Q structure to *truncate* the high bits of the message [KN25].

Definition 4 (RLWE Truncation). *Given $\text{ct} = (b, a) = (-as + \Delta I + \Delta m + e, a) \in \mathcal{R}_{Q_\ell}^2$ with I being an integer polynomial and $\|m\| < 1/2$, the truncating operation is defined as*

$$\text{ct}' = \text{RLWE.Truncate}(\text{ct}) = \text{RLWE.Rescale}_{Q_\ell \rightarrow q_0}(\text{RLWE.MultScalar}(\text{ct}, Q_\ell/\Delta))$$

$$\text{RLWE.Decrypt}_s(\text{ct}') \approx q_0 m + \frac{q_0}{\Delta} e \in \mathcal{R}_{q_0}$$

Fig. 3 shows that the combination of truncation and modulus raising can reset the high part while regaining modulus, and will become a core subroutine in our construction. Other common RLWE operations like Add are defined in the Appendix A.2.

2.3 Approximate CKKS

Aside from the arithmetic triangle mode and boolean flattened mode as our main target encoding discussed in the next section, we sometimes need to *reinterpret* the data in $\mathbb{C}^{N/2}$ as approximate CKKS data.

Definition 5 (CKKS Encoding). *For a message $z \in \mathbb{C}$, fix a scaling factor Δ , its encoding in a complex slot $\mathbb{C}$ in CKKS is*

$$\text{CKKS.Ecd}_{\Delta, \mathbb{C}}(z) = \Delta z$$

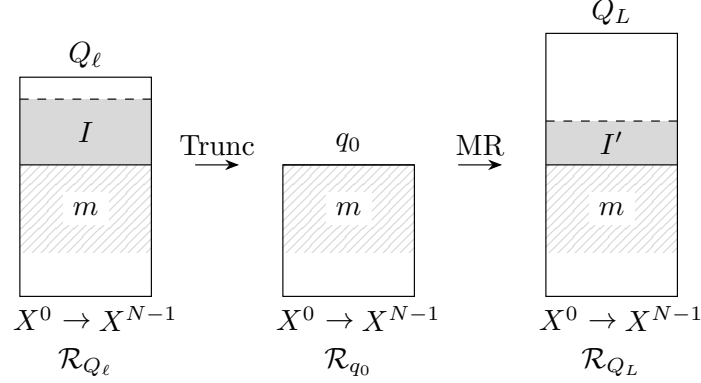


Fig. 3. The RLWE.Truncate and RLWE.ModRaise operations. The input I is truncated and replaced with an I' with possibly smaller norm. The message m is kept. The available modulus is regained to Q_L .

Given a vector of messages $\mathbf{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{C}^{N/2}$, fix a scaling factor Δ , there exists a polynomial $m(X) \in \mathcal{R}_Q = \mathbb{Z}[X]/\langle X^N + 1, Q \rangle$ such that

$$\begin{aligned} m(X) &= \text{CKKS.Ecd}_{\Delta, \mathcal{R}_Q}(\mathbf{z}) \\ m(\zeta_j) &\approx \Delta z_j = \text{CKKS.Ecd}_{\Delta, \mathbb{C}}(z_j) \quad \forall 0 \leq j < N/2 \end{aligned}$$

Such $m(X)$ can be constructed using the inverse transform σ^{-1} subject to the condition that $\|\sigma^{-1}(\Delta \mathbf{z})\| < Q/2$.

We often recognize CKKS encoded data with noise $\mathbf{e}$ as $\Delta \mathbf{z} + \mathbf{e}$. The addition and multiplication of $\mathcal{R}_Q$ give addition and multiplication for complex slots, respectively, through the canonical embedding isomorphism. Therefore CKKS.Sine function can be defined using polynomial approximation (Def. 25) [CHK⁺18].

Using automorphisms in $\mathcal{R}$, operations like CKKS.Rotate and CKKS.Conjugate (Def. 24) can be constructed. With them, linear transformation can be efficiently evaluated to move data between $\mathcal{R}_Q$ and $\mathbb{C}$:

Definition 6 ($\mathcal{R}$ -To- $\mathbb{C}$ and $\mathbb{C}$ -To- $\mathcal{R}$). Given a polynomial $m(X) \in \mathcal{R}_Q$ that is represented as $\mathbf{m} = (m_0, m_1, \dots, m_{N-1}) \in \mathbb{R}^N$. Let $\mathbf{m} = (\mathbf{m}_0; \mathbf{m}_1)$ such that $\mathbf{m}_0 = (m_0, m_1, \dots, m_{N/2-1})$ and $\mathbf{m}_1 = (m_{N/2}, \dots, m_{N-1})$ we can produce two $w_0(X)$ and $w_1(X)$ that $w_0(X) = \text{CKKS.Ecd}_{\Delta, \mathcal{R}_Q}(\mathbf{m}_0)$ and $w_1(X) = \text{CKKS.Ecd}_{\Delta, \mathcal{R}_Q}(\mathbf{m}_1)$ using $\mathbf{V}_{\sigma, 0}$ and CKKS.Conjugate. This is called $\mathcal{R}$ -To- $\mathbb{C}$.

$$\begin{aligned} w_0(X) &= \frac{2}{N} \Re [\text{CKKS.LT}_{\mathbf{V}_{\sigma, 0}}(m(X))] \\ w_1(X) &= \frac{2}{N} \Im [\text{CKKS.LT}_{\mathbf{V}_{\sigma, 0}}(m(X))] \end{aligned}$$

Then by using $w_0(X)$ and $w_1(X)$, we can restore back $m(X)$ using $U_{\sigma,0}$. This is called $\mathbb{C}$ -To- $\mathcal{R}$.

$$m(X) = \text{CKKS.LT}_{U_{\sigma,0}}(w_0(X) + X^{N/2}w_1(X))$$

2.4 Discrete CKKS

Discrete CKKS requires the messages in the complex slots to have discrete shape [BKSS24, AKP25].

Definition 7 (Discrete CKKS Encoding). For a message $m \in \mathbb{Z}_p$ where p is a power-of-two, fix a scaling factor Δ , its encoding in a complex slot $\mathbb{C}$ in discrete CKKS is

$$\text{D-CKKS.Ecd}_{\Delta,\mathbb{C},p}(m) = \Delta \frac{m}{p}$$

Note that m is actually stored in the fractions in the view point of Δ . The general form of discrete CKKS encoded data is $\Delta I + \Delta \frac{m}{p} + e \in \mathbb{C}$ where I is a small integer and e is the noise.

Discrete CKKS allows look-up table LUT evaluation on the m while also cleaning I and e .

Definition 8 (Discrete CKKS LUT Operation). Given $\Delta I + \Delta \frac{m}{p} + e \in \mathbb{C}$, for arbitrary LUT function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, there exists an operation (parameterized by interpolation order κ) that evaluates the LUT while cleaning the I and e when $|I| < K$ and $\frac{ep}{\Delta}$ is small enough.

$$\text{D-CKKS.LUT}_f \left(\Delta I + \Delta \frac{m}{p} + e \right) = \Delta \frac{f(m)}{p} + e' + e''$$

where e' is the noise after noise cleaning with $e' = \frac{\Delta}{p} \cdot O \left(\left| \frac{ep}{\Delta} \right|^{\kappa+1} \right)$, and e'' is the noise caused by homomorphic operations.

We remark that, D-CKKS.LUT supports simultaneous evaluation of multiple LUTs with low costs and this is called *multi-value* LUT evaluation.

3 Arithmetic Triangle Mode

We follow the idea of Flatten operation from [GV25], which generalizes the modulo operation $[\cdot]_t$ from t being an integer to t being a polynomial. While they present the operation as modulo-by-1, we have a top-down narrative:

Definition 9 (The Flatten operation). For a (positive) $m \in \mathbb{Z}$, its binary representation is $m = \sum_{i=0}^{\infty} b_i 2^i$ where $b_i \in \{0, 1\}$. It can be viewed as a polynomial $\sum_{i=0}^{\infty} b_i X^i$ evaluated at $X = 2$ (An isomorphism between $\mathbb{Z}$ and $\mathbb{Z}[X]/\langle X - 2 \rangle$). If we further want to modulo the high bits, by introducing the relation $X^n =$

$X - 2 = t$ that maps the high bits into the ideal $\langle X - 2 \rangle$, we can modulo the ideal to get $\mathcal{Z}_t = \mathbb{Z}[X]/\langle X^n - X + 2, X - 2 \rangle$ and define

$$\begin{aligned} [\cdot]_t : \mathbb{Z} &\rightarrow \mathcal{Z}_t \\ m &\mapsto \sum_{i=0}^{n-1} b_i X^i \end{aligned}$$

Note that the ideal $\langle 2^n \rangle$ in $\mathbb{Z}$ is mapped to the ideal $\langle X - 2 \rangle$ in $\mathcal{Z} = \mathbb{Z}[X]/\langle X^n - X + 2 \rangle$, so $[\cdot]_t$ becomes a ring isomorphism between $\mathbb{Z}_{2^n} = \mathbb{Z}/\langle 2^n \rangle$ and $\mathcal{Z}_t$. Therefore, addition and multiplication in $\mathbb{Z}_{2^n}$ can be carried out in $\mathcal{Z}_t$ if we are able to maintain $\mathcal{Z}_t$ somewhere inside $\mathcal{Z}_{\mathbb{R}} = \mathbb{R}[X]/\langle X^n - X + 2 \rangle$.

In RLWE settings, polynomials are noisy. We follow the idea of CKKS by setting a scaling factor Δ so the resulting $\Delta[m]_t$ can be separated from noise. We now have $\Delta[m]_t$ in $\mathcal{Z}_{\mathbb{R}}$ and the overflow from addition and multiplication is in the ideal $\langle \Delta(X - 2) \rangle$.

On the other hand, the $\langle \Delta(X - 2) \rangle$ ideal is hard to deal with. REFHE introduces modulo-by-ideal operations. We instead follow the idea of GBFV by multiplying by $\frac{1}{t}$ so the $\langle \Delta(X - 2) \rangle$ ideal is turned into an integer ideal $\langle \Delta \rangle$. This gives us $\frac{\Delta}{t}[m]_t \in \mathcal{Z}_{\mathbb{R}}$.

Definition 10 (Encoding in $\mathcal{Z}_{\mathbb{R}}$). For a message $m \in \mathbb{Z}_{2^n}$, fix a scaling factor Δ , the encoding of m in $\mathcal{Z}_{\mathbb{R}}$ is

$$\mathcal{Z}_{\mathbb{R}}.\text{Ecd}_{\Delta,t}(m) = \frac{\Delta}{t}[m]_t = \Delta \left(\frac{m}{2^n} - \sum_{k=0}^{n-1} \frac{X^k}{2^{k+1}} [m]_{0 \leq i \leq k} \right)$$

where $[m]_{0 \leq i \leq k} = \sum_{i=0}^k b_i 2^i$ is the lower $k+1$ bits of m .

As a pictorial illustration, the encoded polynomial looks like a triangle when the $\frac{m}{2^n}$ term in the constant coefficient is not drawn (Fig. 1). We remark that the “essential” bits are in the upper side of the triangle, and noise can contaminate lower part of the triangle.

We generally recognize the encoded polynomial to be of form

$$f = \Delta I + \frac{\Delta}{t}[m]_t + e$$

where I is a polynomial with integer coefficients and e is small noise.

Definition 11 (Decoding in $\mathcal{Z}_{\mathbb{R}}$). Given a polynomial $f = \Delta I + \frac{\Delta}{t}[m]_t + e$ in $\mathcal{Z}_{\mathbb{R}}$, we define the decoding function to be

$$\mathcal{Z}_{\mathbb{R}}.\text{Dcd}_{\Delta,t}(f) = \left\lfloor \frac{t}{\Delta} f \right\rfloor (2) \pmod{2^n}$$

with $\frac{t}{\Delta} f$ happening in $\mathcal{Z}_{\mathbb{R}}$ and “(2)” denoting evaluation at $X = 2$ in $\mathcal{Z}_{\mathbb{R}}$.

We emphasize that the rounding happens before polynomial evaluation as otherwise the noise will amplify through the evaluation.

Theorem 1 (Arithmetic Decoding Correctness). *We can correctly decode if $m \in \mathbb{Z}_{2^n}$, I is a polynomial with integer coefficients, and $\|te\|_\infty < \frac{\Delta}{2}$*

Proof. $\frac{t}{\Delta}f = tI + [m]_t + \frac{t}{\Delta}e$. Since $\|te\|_\infty < \frac{\Delta}{2}$ and I contains integer coefficients, the rounding operation gives $tI + [m]_t$. When evaluated in $\mathcal{Z}_\mathbb{R}$, $[m]_t(2) = m$ and $tI(2)$ is 0.

We remark that for bootstrapping to function correctly, the requirement on the magnitude of the noise is stricter (Theorem 3, 5, and 6).

As long as the noise exceeds the bounds with negligible probability, the decoding fails with negligible probability. Moreover, the CKKS bootstrapping failing probability is also negligible (Prop. 1 and Def. 3). Therefore, we achieve *exact correctness* by appropriate use of software control on the noise (See Sec. 7.1 for more discussion).

Now we discuss the arithmetic of encoded polynomial in $\mathcal{Z}$.

Definition 12 (Addition in $\mathcal{Z}_\mathbb{R}$ is Addition in $\mathbb{Z}_{2^n}$). *For $f, f' \in \mathcal{Z}_\mathbb{R}$ with $f = \Delta I + \frac{\Delta}{t}[m]_t + e$ and $f' = \Delta I' + \frac{\Delta}{t}[m']_t + e'$ with $m, m' \in \mathbb{Z}_{2^n}$, we define the addition operation to be*

$$\mathcal{Z}_\mathbb{R}.\text{Add}(f, f') = f + f' = \Delta I'' + \frac{\Delta}{t}[m + m']_t + e + e'$$

with $I'' = I + I' + C$ where C has integer coefficients being carries propagated from $[m]_t + [m']_t = [m + m']_t + tC$.

Definition 13 (Multiplication in $\mathcal{Z}_\mathbb{R}$ is Multiplication in $\mathbb{Z}_{2^n}$). *For $f, f' \in \mathcal{Z}_\mathbb{R}$ with $f = \Delta I + \frac{\Delta}{t}[m]_t + e$ and $f' = \Delta I' + \frac{\Delta}{t}[m']_t + e'$ with $m, m' \in \mathbb{Z}_{2^n}$, we define the multiplication operation to be*

$$\mathcal{Z}_\mathbb{R}.\text{Mult}(f, f') = \frac{1}{\Delta^2} \cdot \Delta t \cdot f \cdot f' = \Delta I'' + \frac{\Delta}{t}[m \cdot m']_t + e''$$

Note that this closely resembles the BFV multiplication. We rescale twice to divide by Δ^2 as the constant $\frac{t}{\Delta}$ is not representable with small noise in $\mathcal{Z}_\mathbb{R}$, nor is t . Therefore, one full multiplication consumes two CKKS levels. We remark that full multiplications can be fused together and the constant will be Δt^2 or Δt^3 , consuming less levels.

Definition 14 (Multiplication Shortcut in $\mathcal{Z}$). *If we have $g = \Delta[m']_t + e'$ in $\mathcal{Z}_\mathbb{R}$, the multiplication can be simplified to*

$$\mathcal{Z}_\mathbb{R}.\text{MultShort}(f, g) = \frac{1}{\Delta} \cdot f \cdot g = \Delta I'' + \frac{\Delta}{t}[m \cdot m']_t + e''$$

Note that this consumes only one level.

4 The SIMD Homomorphism Chain

We now discuss how to embed $\mathcal{Z}_{\mathbb{R}}$ in $\mathcal{R}_Q$. Rewriting the SIMD homomorphism chain in Fig. 2, we have

$$\mathcal{R}_Q \leftarrow \mathcal{R}_{\mathbb{R}} \cong \mathbb{C}^{N/2} = (\mathbb{C}^{n/2})^{N/n} \cong \mathcal{Z}_{\mathbb{R}}^{N/n} \xleftarrow{\Delta} \mathcal{Z}_t^{N/n}$$

We first demonstrate the case of $N/n = 1$.

Definition 15 (Embedding of $\mathcal{Z}_{\mathbb{R}}$ in $\mathcal{R}_{\mathbb{R}}$). *Given a polynomial $f(X) \in \mathcal{Z}_{\mathbb{R}}$, its embedding in $\mathcal{R}_{\mathbb{R}}$ is*

$$\begin{aligned} \mathcal{R}_{\mathbb{R}}.\text{Embed}_{\mathcal{Z}_{\mathbb{R}}}(f(X)) &= \sigma^{-1}(\tau(f(X))) \\ \mathcal{Z}_{\mathbb{R}}.\text{Recover}_{\mathcal{R}_{\mathbb{R}}}(g(X)) &= \tau^{-1}(\sigma(g(X))) \end{aligned}$$

The rounding operation from $\mathcal{R}_{\mathbb{R}}$ to $\mathcal{R}$ essentially adds rounding error $e \in \mathcal{R}_{\mathbb{R}}$ with each coefficient in $(-1/2, 1/2]$. The correctness of embedding a polynomial from $\mathcal{R}$ in $\mathcal{R}_Q$ requires the coefficients of the polynomial to be smaller than $Q/2$.

Theorem 2 (Correctness of embedding $\mathcal{Z}_{\mathbb{R}}$ in $\mathcal{R}_Q$). *Given a polynomial $f(X) \in \mathcal{Z}_{\mathbb{R}}$ with $\|f\|_{\infty} = B$, its embedding in $\mathcal{R}_Q$ is*

$$\begin{aligned} g(X) &= \mathcal{R}_Q.\text{Embed}_{\mathcal{Z}_{\mathbb{R}}}(f(X)) = \llbracket \sigma^{-1}(\tau(f(X))) \rrbracket_Q \\ h(X) &= \mathcal{Z}_{\mathbb{R}}.\text{Recover}_{\mathcal{R}_{\mathbb{R}}}(g(X)) = \tau^{-1}(\sigma(g(X))) \end{aligned}$$

we have $h(X) \approx f(X)$ with small error if $\|\sigma^{-1}\| \cdot \|\tau\| \cdot B < Q/2$. Let the rounding error be e , the small error is $\tau^{-1}(\sigma(e))$ and its norm is bounded by $\|\tau^{-1}\| \cdot \|\sigma\| / 2$.

The rescaling in $\mathcal{Z}_{\mathbb{R}}$ is implemented by rescaling in $\mathcal{R}_Q$:

Proposition 2 (Rescaling in $\mathcal{R}$ is Rescaling in $\mathcal{Z}$). *Given a $g(X) \in \mathcal{R}_Q$ encoding $f(X) \in \mathcal{Z}_{\mathbb{R}}$ that is encrypted in $\text{ct} = (-as + g + e, a) \in \mathcal{R}_Q^2$, the RLWE.Rescale operation gives noisy rescaling/division in $\mathcal{Z}$:*

$$\begin{aligned} \text{ct}' &= \text{RLWE.Rescale}_q(\text{ct}) \in \mathcal{R}_{Q/q}^2 \\ h(X) &= \text{RLWE.Decrypt}_s(\text{ct}') = \left\lfloor \frac{g(X)}{q} \right\rfloor + I \\ f(X) &= \tau^{-1}(\sigma(h(X))) = \left\lfloor \frac{f(X)}{q} \right\rfloor + \tau^{-1}(\sigma(I)) + e \end{aligned}$$

Where e is a small error for approximating $\tau^{-1} \left(\sigma \left(\left\lfloor \frac{g(X)}{q} \right\rfloor \right) \right)$ with $\left\lfloor \frac{f(X)}{q} \right\rfloor$. The rescaling noise in $\mathcal{Z}_{\mathbb{R}}$ is bounded by $\|\tau^{-1}\| \cdot \|\sigma\| \cdot \|I\|$.

Now we discuss the case of $N/n \neq 1$.

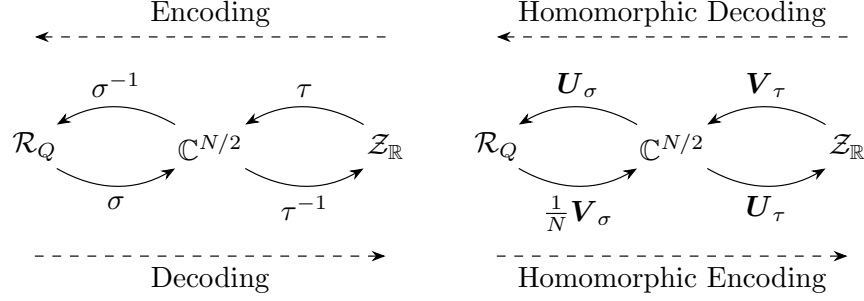


Fig. 4. Encoding/Decoding and Homomorphic Encoding/Decoding. Note that Homomorphic Encoding/Decoding actually evaluate the Decoding and Encoding.

Definition 16 (Embedding of N/n copies of $\mathcal{Z}_{\mathbb{R}}$ in $\mathcal{R}_Q$). Given N/n polynomial $f^{(i)}(X) \in \mathcal{Z}_{\mathbb{R}}$ for $0 \leq i < N/n$ that corresponds to $\mathbf{m} \in \mathbb{Z}_{2^n}^{N/n}$, its embedding in $\mathcal{R}_{\mathbb{R}}$ is

$$\begin{aligned} \mathcal{R}_{\mathbb{R}}.\text{Embed}_{\mathcal{Z}_{\mathbb{R}}, \# \mathcal{Z}_{\mathbb{R}}}(\dots, f^{(i)}(X), \dots) &= \sigma^{-1}(\dots, \tau(f^{(i)}(X)), \dots) \\ \mathcal{Z}_{\mathbb{R}}.\text{Recover}_{\mathcal{R}_{\mathbb{R}}, k}(g(X)) &= \tau^{-1}(\sigma(g(X))_{[kn/2, (k+1)n/2)}) \end{aligned}$$

where $\# \mathcal{Z}_{\mathbb{R}} = N/n$ is the number of $\mathcal{Z}$ -slots.

Given that the k -th $\mathcal{Z}$ -slot corresponds to $[kn/2, (k+1)n/2)$ -th terms in $\mathbb{C}^{N/2}$, we can rotate the $\mathcal{Z}$ -slots using CKKS.Rotate .

Definition 17 (Rotation of $\mathcal{Z}$ -slots). For a vector of messages $\mathbf{m} = (m_0, m_1, \dots, m_{N/n-1}) \in \mathbb{Z}_{2^n}^{N/n}$ that is encoded as $f^{(i)}(X) \in \mathcal{Z}_{\mathbb{R}}$ and embedded in $\mathcal{R}_Q$ as $g(X)$, the rotation of $\mathbf{m}$ can be implemented as

$$\# \mathcal{Z}_{\mathbb{R}}.\text{Rotate}_k(g(X)) = \text{CKKS.Rotate}_{k \cdot \frac{n}{2}}(g(X))$$

such that the result $h(X)$ corresponds to $\mathbf{m}' = (m_k, m_{k+1}, \dots, m_{N/n-1}, m_0, \dots)$.

Now we can discuss moving between $\mathcal{R}_{\mathbb{R}}$ and $\mathcal{Z}_{\mathbb{R}}$. Similar to the linear transformation in the approximate CKKS section, we define

Definition 18 ($\mathcal{Z}$ -To- $\mathbb{C}$ and $\mathbb{C}$ -To- $\mathcal{Z}$). Given a polynomial $f(X) \in \mathcal{Z}_{\mathbb{R}}$ that is represented as $\mathbf{f} = (f_0, f_1, \dots, f_{n-1}) \in \mathbb{R}^n$. Let $\mathbf{f} = (\mathbf{f}_0; \mathbf{f}_1)$ and let $g(X) = \mathcal{R}_{\mathbb{R}}.\text{Embed}_{\mathcal{Z}_{\mathbb{R}}}(f(X))$, we can produce two $h_0(X)$ and $h_1(X)$ in $\mathcal{R}_{\mathbb{R}}$ that $\sigma(h_0(X)) = (f_0, f_1, \dots, f_{n/2-1}) \in \mathbb{C}^{n/2}$ and $\sigma(h_1(X)) = (f_{n/2}, \dots, f_{n-1}) \in \mathbb{C}^{n/2}$ using $\mathbf{V}_{\tau,0}$ and $\mathbf{V}_{\tau,1}$. This is called $\mathcal{Z}$ -To- $\mathbb{C}$.

$$\begin{aligned} h_0(X) &= 2\Re[\text{CKKS.LT}_{\mathbf{V}_{\tau,0}}(g(X))] \\ h_1(X) &= 2\Re[\text{CKKS.LT}_{\mathbf{V}_{\tau,1}}(g(X))] \end{aligned}$$

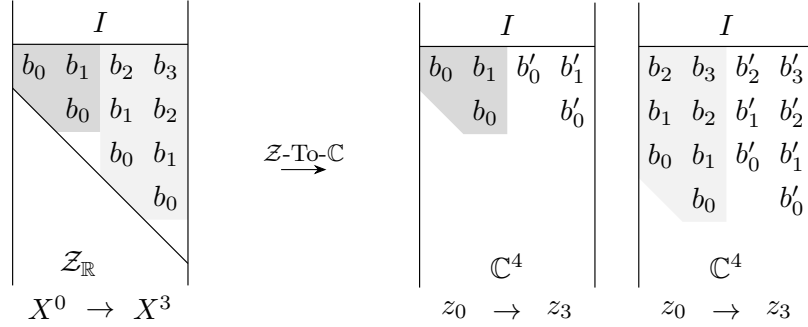


Fig. 5. The $\mathcal{Z}$ -To- $\mathbb{C}$ linear transformation for a fully packed ciphertext. It produces two ciphertexts each containing one half of the original $\mathcal{Z}$ -coefficients in the $\mathbb{C}$ -slots. The example parameter is $n = 4$ and $N = 8$ so there are two $\mathcal{Z}$ -slots, and the data b'_i for another $\mathcal{Z}$ -slot is not drawn here. $X^0 \rightarrow X^3$ means the coefficients of $\mathcal{Z}_{\mathbb{R}}$ while $z_0 \rightarrow z_3$ means the slots in $\mathbb{C}^4$

Then by using $h_0(X)$ and $h_1(X)$, we can restore back $g(X)$ in $\mathcal{R}_{\mathbb{R}}$ using $U_{\tau,0}$ and $U_{\tau,1}$, thus recovering $f(X)$ in $\mathcal{Z}_{\mathbb{R}}$. This is called $\mathbb{C}$ -To- $\mathcal{Z}$.

$$g(X) = \text{CKKS.LT}_{U_{\tau,0}}(h_0(X)) + \text{CKKS.LT}_{U_{\tau,1}}(h_1(X))$$

For N/n copies of $\mathcal{Z}_{\mathbb{R}}$, the linear transformations used are actually

$$U_{\tau,0,\#\mathcal{Z}_{\mathbb{R}}} = I_{\#\mathcal{Z}_{\mathbb{R}}} \otimes U_{\tau,0} = \begin{bmatrix} U_{\tau,0} & & \\ & U_{\tau,0} & \\ & & \ddots \end{bmatrix}$$

and similar matrices defined for $U_{\tau,1}$, $V_{\tau,0}$ and $V_{\tau,1}$. In these matrices, only $n-1$ diagonals are non-zero and therefore the cost of its evaluation is $O(2\sqrt{2n})$ rotations by using baby-step-giant-step strategy for $(n/2-1)$ positive and $(n/2-1)$ negative diagonals. We also give an illustration in Fig. 5 on how $\mathcal{Z}$ -To- $\mathbb{C}$ can break the bits of one ciphertext $g(X)$ into two ciphertexts $h_0(X)$ and $h_1(X)$ and we call it the “breaking-into-halves” effect.

Combined with the linear transformation in approximate CKKS, we are able to define $\mathcal{Z}$ -To- $\mathcal{R}$ and $\mathcal{R}$ -To- $\mathcal{Z}$. The overall encoding/decoding and linear transformation process is also illustrated in Fig. 4.

5 Bootstrapping: Refreshing and Conversion

In this section, we use “refreshing” for bootstrapping inside one mode, while using “conversion” for mode conversion where bootstrapping is usually the backbone.

The goal of bootstrapping is to allow more computation to carry on. The current ciphertext may reach a status that (1) the modulus Q is too small where

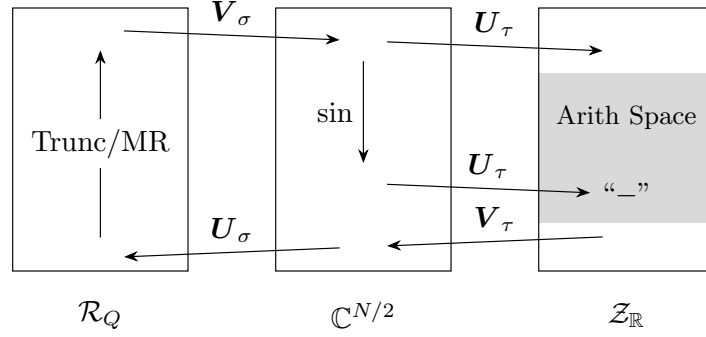


Fig. 6. The Arithmetic-To-Arithmetic Refreshing

further regular computation cannot be applied, or (2) the noise e is too large where further regular computation might corrupt the message.

For the modulus Q problem, the RLWE.ModRaise operation can regain a large amount of modulus, but it also introduces overflow I in the high bits. Therefore, CKKS.Sine and D-CKKS.LUT were devised to clean the overflow. Note that high bits I are integers in the ring $\mathcal{R}_Q$, whereas cleaning happens in $\mathbb{C}$. So $\mathcal{R}$ -To- $\mathbb{C}$ is utilized to move the overflow into $\mathbb{C}$, while also moving the payload message m . The message m needs to be moved back using $\mathbb{C}$ -To- $\mathcal{R}$ when finished.

The noise e problem is trickier. In approximate CKKS, the noise is part of the message. Discrete CKKS, on the other hand, handles discrete data and need clear separation of the noise, therefore D-CKKS.LUT is designed with different interpolation order to clean the noise. We emphasize that D-CKKS.LUT requires a large “gap” between $\frac{\Delta}{p}m$ and e for the noise cleaning functionality to be effective (e.g., [AKP25, Fig. 2]).

5.1 Arithmetic-To-Arithmetic (A2A) Refreshing (Fig. 6)

In arithmetic mode, we are given $f(X) = \Delta I + \frac{\Delta}{t}[m]_t + e$ in $\mathcal{Z}_{\mathbb{R}}$. Our goal is to obtain a new $f'(X) = \Delta I' + \frac{\Delta}{t}[m]_t + e'$ with $\|e'\|$ smaller than $\|e\|$ and $\|I'\|$ usually smaller than $\|I\|$. Note that we do not require cleaning the ΔI completely, which is the key to efficient refreshing.

First, the familiar high I structure reminds us of RLWE.ModRaise. By using (1) $\mathcal{Z}$ -To- $\mathcal{R}$, (2) RLWE.Truncate and ModRaise, and (3) $\mathcal{R}$ -To- $\mathcal{Z}$, we are able to reset I to a new I' with I' being the small noise in Prop 1. This also regains modulus Q and allows more computation to go on. This is called A2A I .

For the noise e , the line META-BTS and Bootstrapping for BFV/GBFV using CKKS [BCC+22, KSS24, Kim25a] gives us the inspiration that we can bootstrap the noise e alone. Note that if we multiply $f(X)$ by t in $\mathcal{Z}_{\mathbb{R}}$, the message $\frac{\Delta}{t}[m]_t$ also becomes a part of the high I structure (Fig. 10), and can be cleaned by RLWE.Truncate. Given te , we are able to bootstrap it using CKKS.Sine to regain modulus, then multiplying by t^{-1} in $\mathcal{Z}_{\mathbb{R}}$ gives us the e . Subtracting the

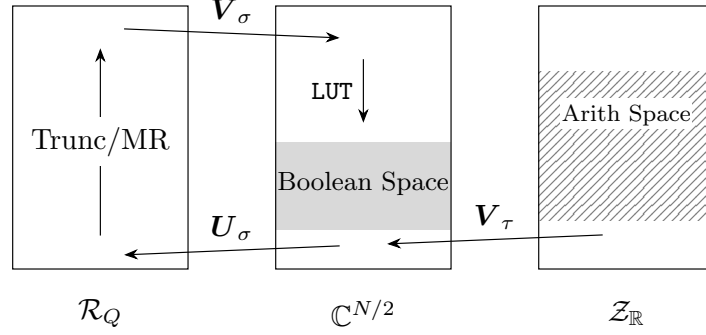


Fig. 7. The Arithmetic-To-Boolean Conversion

bootstrapped e from the original ciphertext, we obtain e' that is smaller. It can be summarized as (1) $\mathcal{Z}$ -To- $\mathcal{R}$ that fuses multiplication of t , (2) RLWE.Truncate and ModRaise (3) $\mathcal{R}$ -To- $\mathbb{C}$, (4) CKKS.Sine (5) $\mathbb{C}$ -To- $\mathcal{Z}$ that fuses multiplication by t^{-1} in $\mathcal{Z}_{\mathbb{R}}$ and (6) Subtraction from the original ciphertext. If the precision offered by CKKS.Sine is limited, we can repeat such process as each bootstrapping reduces the noise until it could not be cleaned further. This is called A2A e .

For the full A2A refreshing, we first execute A2A I to reset the large I to a smaller one, then execute A2A e to clean the noise. It is not recommended to reverse the order of them because large I during A2A e will introduce additional noise from linear transformation and will have less noise cleaning effect.

It is worth noting that the arithmetic overflow I is a major cause for noise growth in multiplication like $\mathcal{Z}_{\mathbb{R}}.\text{Mult}$. We may sometimes only invoke A2A I to reduce noise growth of further multiplications, especially when the operands of multiplication have asymmetric noise/overflow.

5.2 Arithmetic-To-Boolean (A2B) Conversion (Fig. 7)

The arithmetic mode gives us $f(X) = \Delta I + \frac{\Delta}{t}[m]_t + e$ in $\mathcal{Z}_{\mathbb{R}}$ with $[m]_t = \sum_{i=0}^{n-1} b_i X^i$ where b_i is the i -th bit of $m \in \mathbb{Z}_{2^n}$. The boolean mode requires $\Delta \mathbf{b} + e = \Delta(b_0, b_1, \dots, b_{n-1}) + e$ in $\mathbb{C}^n$.

As the arithmetic mode has a triangle shape (Fig. 1) for $\frac{\Delta}{t}[m]_t$, the conversion is the extraction of the bits in upper side of the triangle. This can be achieved by D-CKKS.LUT with the target function $\text{LUT}_{\text{MSB}} : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$ that extracts the most significant bit.

First we discuss the case where $2^n \leq p$, then discuss how to iteratively achieve arithmetic-to-boolean conversion for large 2^n .

For the case $2^n \leq p$, $\frac{\Delta}{t}[m]_t$ can be reinterpreted as discrete CKKS data if viewing polynomial as vector.

$$\begin{aligned} \frac{\Delta}{t}[m]_t &= \Delta \left(\frac{m}{2^n} - \sum_{k=0}^{n-1} \frac{X^k}{2^{k+1}} [m]_{0 \leq i \leq k} \right) \\ &\stackrel{\text{cast}}{=} - \left(\dots, \frac{\Delta}{2^n} \cdot \frac{1}{2} \cdot 2^{n-k} [m]_{0 \leq i \leq k}, \dots \right) + \left(\frac{m}{2^n}, 0, 0, \dots \right) \\ &= -(\dots, \Delta \overline{b_k b_{k-1} \dots}, \dots) + \left(\Delta \frac{m}{2^n}, 0, 0, \dots \right) \end{aligned}$$

where $\overline{b_k b_{k-1} \dots}$ denoting $\frac{1}{2}b_k + \frac{1}{4}b_{k-1} + \dots$.

We need a small preprocessing that removes $\Delta \frac{m}{2^n}$. Note that the constant coefficient contains $\Delta \frac{m}{2^n} - \Delta \overline{b_0}$ and the first coefficient contains $-\Delta \overline{b_1 b_0}$. We can replace the constant coefficient with twice of the first coefficient, so that the constant coefficient becomes $-\Delta \overline{b_1 b_0} = \Delta I - \Delta \overline{b_0}$. This preprocessing is merged into the $\mathcal{Z}$ -To- $\mathbb{C}$ linear transformation and is called special- b_0 $\mathcal{Z}$ -To- $\mathbb{C}$.

The D-CKKS.LUT operation is level-consuming and is often carried out during bootstrapping. Therefore the conversion procedure is (1) Special- b_0 $\mathcal{Z}$ -To- $\mathcal{R}$, (2) RLWE.Truncate and ModRaise, (3) $\mathcal{R}$ -To- $\mathbb{C}$, (4) D-CKKS.LUT that evaluates LUT_{MSB} .

The $2^n > p$ case needs more iteration. Let $w = \log p$ and $d = \lceil n/w \rceil$. We need d iterations to extract $\overline{b_{n-1}}$. Note that the coefficient in the $(n-1)$ -th coefficient is

$$\begin{aligned} &\Delta \overline{b_{n-1} \dots b_{(d-1)w} \dots b_{w-1} \dots b_0} \\ &= \Delta \overline{b_{n-1} \dots b_{(d-1)w}} + \frac{\Delta}{p} \overline{b_{(d-1)w-1} \dots b_{(d-2)w}} + \dots + \frac{\Delta}{p^{d-1}} \overline{b_{w-1} \dots b_0} \end{aligned}$$

We can iteratively extract $\Delta \overline{b_{w-1} \dots b_0}$ using LUT_{ID} that evaluates the identity function until we obtain $\Delta \overline{b_{n-1} \dots b_{(d-1)w}}$. The iterative process is also shown in Fig. 8.

The conversion procedure is summarized as (1) Special- b_0 $\mathcal{Z}$ -To- $\mathbb{C}$ to obtain one *core ciphertext* (with two ciphertexts due to Fig. 5), (2) Pick the least w bits from core ciphertext by masking and execute one iteration: [1] $\mathbb{C}$ -To- $\mathcal{R}$, [2] RLWE.Truncate and ModRaise, [3] $\mathcal{R}$ -To- $\mathbb{C}$, [4] D-CKKS.LUT that evaluates both LUT_{MSB} and LUT_{ID} , and [5] Subtract LUT_{ID} result from core ciphertext. (3) Do the next iteration with the next w bits. (4) When all iterations are finished, combine all ciphertexts obtained using LUT_{MSB} .

One concern is that in each subtraction step, the number of rotation grows linearly with the bit-width n . We therefore set a threshold A2B-cutoff where bits below the threshold are treated as noise for D-CKKS.LUT, so the number of rotations is constant⁴ for one ciphertext in each subtraction step and the overall number of rotations for subtraction is $O(n)$.

⁴ For A2B-cutoff = -16 and $w = 4$, the number of rotation is 3.

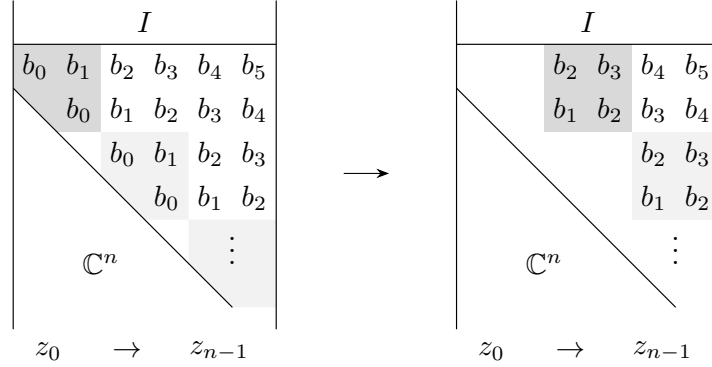


Fig. 8. Iterative process of extracting w -MSB and removing w -LSBs in Arithmetic-To-Boolean Conversion. We set $w = 2$ here for exposition and is not the actual parameter. The thicker gray region is the extracted w -MSB, while the thinner gray region is the removed w -LSBs. The arrow in the middle means next iteration. $z_0 \rightarrow z_{n-1}$ means the slots in $\mathbb{C}^n$. The “breaking-into-halves” effect of $\mathcal{Z}$ -To- $\mathcal{C}$ in Fig. 5 is not pictured here.

Batched Arithmetic-To-Boolean (B-A2B) Conversion We further notice that, inside each iteration, only w bits of each n -wide slot is utilized, therefore in each iteration we can bootstrap n/w ciphertexts at once, and the number of CKKS bootstrapping is amortized to *one per ciphertext*.

The conversion procedure for n/w ciphertexts is summarized as (1) Special- b_0 $\mathcal{Z}$ -To- $\mathcal{C}$ to obtain n/w *core ciphertexts* ($2(n/w)$ ciphertexts), (2) (Pre-)rotate the i -th core ciphertext by $-i \cdot w$ for later merging, (3) Pick the least w bits from n/w core ciphertexts using *masking*, merge them into two CKKS ciphertexts using addition and execute one iteration: [1] $\mathbb{C}$ -To- $\mathcal{R}$, [2] RLWE.Truncate and ModRaise, [3] $\mathcal{R}$ -To- $\mathcal{C}$, [4] D-CKKS.LUT that evaluates both LUT_{MSB} and LUT_{ID} , [5] Subtract LUT_{ID} results from each core ciphertext, with corresponding masking. (4) Do the next iteration with the next w bits. (5) When all iterations are finished, *combine* LUT_{MSB} results into n/w ciphertexts using masking, and (post-)rotate the i -th result ciphertext by $i \cdot w$ to restore to the original offset.

Linear transformation $\mathbb{C}$ -To- $\mathcal{R}$ can process two ciphertexts at once by merging the messages in the second ciphertext into the imaginary part of the first ciphertext. The subtraction of LUT_{ID} results and the combination of LUT_{MSB} results consumes the same CKKS level, as they are independent branches.

For complexity analysis, for each core ciphertext, pre-rotation and post-rotation each takes 1 rotation; the subtraction step in each iteration takes constant number of rotations due to the cutoff parameter. The number of rotations for pre-rotation/subtraction/post-rotation for each core ciphertext is $O(n)$.

5.3 Boolean-To-Boolean (B2B) Refreshing (Fig. 9)

The boolean-to-boolean refreshing is discrete CKKS bootstrapping for bits and is implemented using polynomial approximation of cosine [AKP25]. For complete-

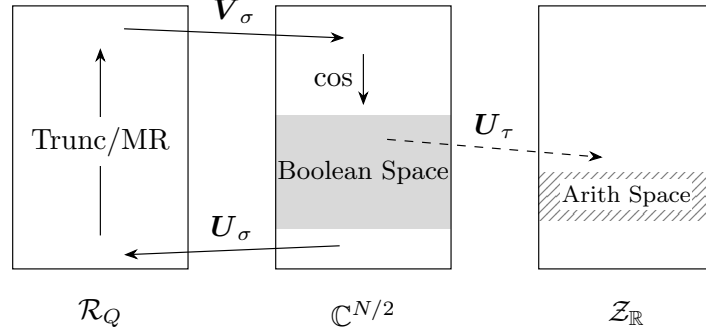


Fig. 9. The Boolean-To-Boolean Refreshing (solid arrows) and the Boolean-To-Arithmetic Conversion (dashed arrow).

ness, we describe it as (1) $\mathbb{C}$ -To- $\mathcal{R}$, (2) RLWE.Truncate and RLWE.ModRaise , (3) $\mathcal{R}$ -To- $\mathbb{C}$ and (4) D-CKKS.LUT that is implemented as cosine evaluation.

5.4 Boolean-To-Arithmetic (B2A) Conversion (Fig. 9)

Given a ciphertext containing $\Delta \mathbf{b} = (b_0, \dots, b_{n-1})$ in $\mathbb{C}^n$, it can be viewed as $\Delta[m]_t$ if linear transformed to $\mathbb{Z}_R$. So the conversion procedure can be fused with a $\mathbb{Z}_R.\text{Mult}$ operation as is (1) $\mathbb{C}$ -To- $\mathbb{Z}$ (2) $\mathbb{Z}_R.\text{MultShort}$. If the next operation for m is $\mathbb{Z}_R.\text{Add}$, we can directly call a $\mathbb{C}$ -To- $\mathbb{Z}$ that fuses the multiplication of t^{-1} .

6 Whole Picture of Our Scheme

We summarize operations needed and offered by our scheme for ease of reference.

- RLWE namespace : KeyGen , Encrypt , Decrypt , Add , Tensor , Automorphism , KeySwitch , ModRaise , Rescale , Truncate .
- CKKS namespace : $\text{Ecd} : \mathbb{C}^{N/2} \rightarrow \mathcal{R}_Q$, Dcd , Sine , Rotate , Conjugate , LT , Add , Mult
- D-CKKS namespace : $\text{Ecd} : \mathbb{Z}_p \rightarrow \mathbb{C}$, Dcd , LUT .
- $\text{Arithmetic } \mathbb{Z}_R \text{ namespace}$: $\text{Ecd} : \mathbb{Z}_{2^n} \rightarrow \mathbb{Z}_R$, Dcd , Add , Mult , MultShort .
- $\text{Many arithmetic } \# \mathbb{Z}_R \text{ namespace}$: Rotate .
- $\text{Homomorphism namespace}$: $\text{Embed} : \mathbb{Z}_R \rightarrow \mathcal{R}_Q$ and $\text{Recover} : \mathcal{R}_Q \rightarrow \mathbb{Z}_R$.
- $\text{Boolean } \mathbb{C} \text{ namespace}$: $\text{Ecd} : \mathbb{Z}_{2^n} \rightarrow \mathbb{C}^n$, Dcd , AND , OR , NOT , bit shifting/rotation, sign extraction, and many others. Boolean-To-Arithmetic conversion (B2A).
- $\text{Bootstrapping namespace}$: $\text{Arithmetic-To-Arithmetic (A2A)}$ for I , $\text{A2A } e$, $\text{Arithmetic-To-Boolean (A2B)}$, Batched A2B , $\text{Boolean-To-Boolean (B2B)}$.

Each Add and Mult have ciphertext-ciphertext and ciphertext-plaintext versions; boolean operations with two operands also have two versions. Ciphertext-plaintext $\mathbb{Z}_R.\text{Mult}$ is implemented using $\mathbb{Z}_R.\text{MultShort}$.

Table 3. Empirical noise growth (in bits).

BitW	n	Mult (max/avg)	MultShort	BitW	n	Mult	MultShort
	8	4.3 / 3.9	2.6 / 1.5	64	5.6 / 5.0	3.3 / 2.6	
	16	5.0 / 4.3	2.4 / 1.7	128	5.8 / 5.4	3.6 / 2.9	
	32	4.9 / 4.6	2.9 / 2.3	256	6.3 / 5.9	4.0 / 3.4	
Arith		$e_A^\dagger = -24.4$	Arith	$I_A^\ddagger = 2.8$	Bool		$e_B^\dagger = -20.4$

[†] The value of $\log(\|e\|/\Delta)$

[‡] The value of $\log\|I\|$

7 Analysis and Parameter Selection

7.1 Noise Analysis and Security Implications

The security notions of FHE are *unconventional* as FHE is inherently dynamic by allowing evaluation of arbitrary user-chosen circuits. From the attack [LM21, LMSS22] that proposes the IND-CPA^D security, to recent attacks like [GNSJ24, CCP⁺24, CSBB24] that incur discussion like *application-aware* IND-CPA^D security [AAMP26] stating that software controls should be mandated based on user applications, to the very recent attack stating the impossibility of IND-CPA^D security for certain schemes [CCRS26], this research area is actively evolving. Among these attacks, a critical connection is the interchangeability between *security* and *correctness*. In short, if the scheme does not achieve correctness, its security is also corrupted, independent of the (R)LWE assumption.

We stick to the *application-aware* security model, relying on software control (e.g., application specification language in [AAMP26], compilers like [ACG⁺25, EGR25]) to provide *exact correctness* and *application-aware* IND-CPA^D security.

In this section, we provide worst-case noise analysis tools as the basis for software control. Rigorous average-case noise analyses [KPZ21, CCH⁺24, BCM⁺25, BJSW25, GZ25] can be used in production with proper validation and are out of the scope of this work. We provide *debugging* utilities in our open source prototype that can extract the current noise upon decryption for examination of the noise behavior.

Arithmetic Mode. We want messages after A2A refreshing to maintain the form $\Delta I_A + \frac{\Delta}{t}[m]_t + e_A$ in $\mathcal{Z}_{\mathbb{R}}$, where e_A is an *arithmetic bottom noise* that cannot be reduced by further bootstrapping. The *minimal overflow* I_A as the result of A2A I satisfies $\|I_A\| < 16$ with negligible failing probability because we use sparse encapsulated secret key during ModRaise.

Theorem 3 (A2A Refreshing Correctness). *To refresh an arithmetic mode ciphertext with messages $\Delta I + \frac{\Delta}{t}[m]_t + v$ to bottom noise, we require the input noise v to satisfy*

$$C_1 \left(\frac{\|tv\|}{\Delta} \right)^3 < \frac{\|e_A\|}{\Delta}$$

where C_1 is a constant determined by CKKS.Sine.

Proof. During A2A e, we are bootstrapping the noise to in $\mathbb{C}$. CKKS.Sine incurs additional error of $C_1 \Delta (\|v\| / \Delta)^3$. We require the additional noise to be smaller than the bottom noise.

We may use iterative CKKS bootstrapping like META-BTS [BCC⁺22] to further loosen the requirement on the input noise.

For multiplication in arithmetic mode, we first examine the evolution of polynomial norm in $\mathcal{Z}$:

Proposition 3 (Expansion in $\mathcal{Z}$, [BFG⁺25]). For $f, g \in \mathcal{Z}_{\mathbb{R}}$ and $t = X - 2$, we have

$$\begin{aligned}\|f + g\| &\leq \|f\| + \|g\| \\ \|f \cdot g\| &\leq 3n \cdot \|f\| \cdot \|g\| \\ \|t \cdot f\| &\leq 4 \|f\|\end{aligned}$$

The $3n$ here is the *worst-case* expansion factor in $\mathcal{Z}_{\mathbb{R}}$.

Theorem 4 (Arithmetic Multiplication Noise Growth). For two messages $f = \Delta I + \frac{\Delta}{t}[m]_t + v$ and $f' = \Delta I' + \frac{\Delta}{t}[m']_t + v'$, the noise v'' and the overflow I'' after $\mathcal{Z}_{\mathbb{R}}.\text{Mult}$ satisfies

$$\begin{aligned}\|v''\| &\leq 3n \left[\max(4 \|I\|, 1) \cdot \|v'\| + \max(4 \|I'\|, 1) \cdot \|v\| + \frac{4 \|v\| \|v'\|}{\Delta} \right] \\ \|I''\| &\leq 3n (4 \|I\| \cdot \|I'\| + \|I\| + \|I'\| + 1)\end{aligned}$$

For the multiplication of f and $g = \Delta[m_g]_t + v_g$, the noise v''' and the overflow I''' after $\mathcal{Z}_{\mathbb{R}}.\text{MultShort}$ satisfies

$$\begin{aligned}\|v'''\| &\leq 3n \left[\max(\|I\|, 1) \cdot \|v_g\| + \|v\| + \frac{\|v\| \|v_g\|}{\Delta} \right] \\ \|I'''\| &\leq 3n (\|I\| + 1)\end{aligned}$$

Proof. We have $\|[m]_t\| \leq 1$ and $\left\| \frac{[m]_t}{t} \right\| \leq 1$, and

$$\begin{aligned}v'' &= t(I \cdot v' + I' \cdot v) + [m]_t \cdot v' + [m']_t \cdot v + \frac{t}{\Delta} v \cdot v' \\ v''' &= I \cdot v_g + \frac{1}{t}[m]_t \cdot v_g + \frac{1}{t}[m_g]_t \cdot v + \frac{1}{\Delta} v \cdot v_g\end{aligned}$$

Boolean Mode. We want to maintain ciphertexts after B2B refreshing to have messages of form $\Delta \mathbf{b} + \mathbf{e}_B$, where $\mathbf{e}_B$ is the *boolean bottom noise*. For boolean operations, the underlying operation is multiplication in $\mathbb{C}$, so the norm evolution is $\|f \cdot f'\| \leq \|f\| \cdot \|f'\|$. We now state the correctness requirement for bootstrapping operations.

Theorem 5 (B2B Refreshing Correctness). *To refresh a boolean mode ciphertext with messages $\Delta \mathbf{b} + \mathbf{v}$ to bottom noise, we require*

$$C_2 \left(\frac{\|\mathbf{v}\|}{\Delta} \right)^2 < \frac{\|\mathbf{e}_B\|}{\Delta}$$

where C_2 is a constant determined by CKKS.Cosine.

Proof. Cosine evaluation will reduce the noise to $C_2 \Delta (\|\mathbf{v}\| / \Delta)^2$, and we require it to be smaller than bottom noise.

Theorem 6 (A2B Conversion Correctness). *To convert an arithmetic mode ciphertext with messages $\Delta I + \frac{\Delta}{t}[m]_t + \mathbf{v}$ to boolean mode, we require the input noise $\mathbf{v}$ to satisfy*

$$C_3 \left(\frac{\|\mathbf{v}\| p}{\Delta} \right)^{\kappa+1} < \frac{\|\mathbf{e}_B\| p}{\Delta}$$

where C_3 is a constant determined by D-CKKS.LUT.

Proof. See Def. 8 for the analysis.

For illustration purpose, we conducted experiments to record the empirical bottom noise for both arithmetic mode and boolean mode, and record the empirical noise growth of two kinds of multiplications *for bottom ciphertexts* in arithmetic mode with the parameter in Table 5. The results are shown in Table 3.

7.2 Level Analysis

Let L be the available multiplicative depth for CKKS parameter selection. The overall level consumption for each bootstrapping, and the available arithmetic or boolean spaces are presented in Table 4. For $\mathcal{R}$ -To- $\mathbb{C}$ and $\mathbb{C}$ -To- $\mathcal{R}$, which corresponds to conventional **CoeffsToSlots** and **SlotsToCoeffs**, we allocate 3 and 2 levels, respectively, for reducing the linear transformation evaluation cost [CCS19]. The CKKS.Sine operation, adapted from [BMTH21], requires a polynomial approximation of degree $d_{g_0} = 32$ and takes $L_{\text{sin}} = 6 + 3$ levels, where 3 comes from three application of double angle formula.

The D-CKKS.LUT operation with interpolation order 1 for $p = 2^w$ where $w = 4$, adapted from [AKP25], requires a polynomial approximation of degree $d_{\text{exp}} = 46$ for the exponential function, and another polynomial evaluation of degree $d = p - 1 = 15$ with interpolation order of 1, consuming $6 + 2 + 5$ levels, where 2 comes from two application of double angle formula.

The cosine operation requires a polynomial approximation of degree $d_{\text{cos}} = 50$ and consumes $6 + 2$ level, where the extra 2 comes from one application of double angle formula and one squaring to obtain $\cos^2(\pi x)$.

Table 4. The level consumption of each bootstrapping.

A2A I	R2C 3		C2Z 1	Avail Arith $L - 7$	Z2C 1	C2R 2
A2A e	R2C 3	Sine 9	C2Z(t^{-1}) 1	Avail Arith $L - 16$	Z2C(t) 1	C2R 2
B-A2B	R2C 3	LUT 13	Comb [†] 1	Avail Bool [‡] $L - 19$	Mask [*] 1	C2R 2
B2B	R2C 3	Cosine 8		Avail Bool $L - 13$		C2R 2

* Ciphertexts from arithmetic mode need 1 level to arrive here using special- b_0 Z2C.

† This level is used for the subtraction of LUT_{ID} results from core ciphertexts, and the combination of LUT_{MSB} results into output ciphertexts, which are independent branches and can be done in the same level. In A2B, the combination step does not consume level and the available level is $L - 18$.

‡ After B-A2B, the level reserved for mask can be used for computation.

7.3 Parameter Selection

We select $L = 20$ for all bootstrapping operations to function, and we select $\log_2 \Delta \approx 43$ for all experiments to carry out, therefore there are 21 primes in the moduli chain. We select 7 auxiliary primes (50-bit) to construct P with $\text{dnum} = 3$ for hybrid key switching. The auxiliary prime is larger than Δ by 6 bits for sparse encapsulated key switching in **ModRaise** to introduce negligible noise. The threshold A2B-cutoff parameter for A2B is set to -16 .

Under such selection, the RLWE modulus is $\log QP \approx 1254$ and we therefore select the RLWE ring dimension to be $N = 2^{16}$. The RLWE error distribution is Gaussian with standard error $\sigma = 3.2$. We use a sparse secret key of Hamming weight $h = 192$ and a sparse encapsulated secret key of Hamming weight $\tilde{h} = 32$. This achieves 128-bit security according to [BTH22].

We employ the level-specific scaling factor Δ_ℓ approach from [KPP22] for eliminating the scaling factor approximation noise. We slightly alter its prime selection logic by setting the starting scaling factor to the first modulus q_0 as **RLWE.Truncate** operation resets the rescaling factor to q_0 and **RLWE.ModRaise** makes it become the starting scaling factor.

We would like to mention that it is also possible to select a uniform ternary main secret key while using the sparse encapsulated key for **ModRaise**, or stick to a single uniform ternary secret key, for approaching the standard recommendation like [ACC⁺18, BCC⁺24]. Our scheme is functional under all these settings, except with a larger noise growth in the former case, or with non-negligible CKKS bootstrapping failing probability and performance penalty (at most 25% by estimation in [AKP25]) in the latter case.

Table 5. Parameters for all experiments

RingDim	Scaling Factor	MulDepth [†]	RLWE Modulus	SK Dist
N	$\log \Delta$	L	$\log QP$	$(h ; \tilde{h})$
2^{16}	43	20	1254	(192 ; 32)

[†] Refers to the CKKS multiplicative depth, not the available multiplicative depth for arithmetic mode in our scheme.

Table 6. Detailed performance numbers of each bootstrapping in our scheme.

n	#Slot	A2A I	A2A [†]	A2B	B-A2B [‡]	B2B	B2A
8	8192	4.3s/0.5ms	10.1s/1.2ms	10.1s/1.2ms	13.0s/0.8ms	6.0s/0.7ms	0.7s/0.1ms
16	4096	4.7s/1.1ms	10.9s/2.7ms	20.5s/5.0ms	27.0s/1.7ms	5.9s/1.4ms	1.1s/0.3ms
32	2048	5.3s/2.6ms	11.8s/5.8ms	40.8s/20ms	57.7s/3.5ms	5.9s/2.9ms	1.7s/0.8ms
64	1024	6.4s/6.3ms	13.5s/13.2ms	80.5s/79ms	128s/ 7.8ms	5.9s/5.8ms	2.8s/2.7ms
128	512	7.7s/15ms	15.3s/29.9ms	164s/320ms	309s/18.9ms	5.8s/11ms	4.1s/8.0ms
256	256	10.2s/40ms	19.5s/76.1ms	327s / 1.3s	823s / 50ms	5.9s/23ms	6.4s/25ms

[†] A2A e data can be obtained by subtracting A2A data with A2A I data.

[‡] Batched A2B processes $n/4$ ciphertexts for bit-width of n .

8 Implementation, Benchmark, and Comparison

We implement our scheme based on OpenFHE [BAB⁺22] v1.4.2 with HEXL [BKS⁺21]. Our experiments were conducted in a single-thread setting on a Linux machine with AMD RYZEN AI MAX+ 395 CPU and 128GB RAM, running Debian 13. The compiler used is clang 19.

The detailed performance of bootstrapping operations for bit-width from 8 to 256 is present in Table 6. The data were collected by executing each bootstrapping 5 times and taking the average, with one extra warm-up run before the collection. A performance comparison of our scheme with previous works can be checked in Table 2.

The experiments for TFHE-rs were conducted in a single thread setting on the same machine with AVX512 acceleration, using `integer/bench.rs`, especially `mul_parallelized` from the commit 222a7e93c under `RAYON_NUM_THREADS=1`.

Compared with the CPL scheme [CPL25], our multiplication takes 2 CKKS bootstrapping operations (or less for leveled execution), while their scheme takes 3 to 5 CKKS bootstrapping operations; we also concretely outperform their scheme by $3\times$ to $3.8\times$ for multiplication for bit-width from 64 to 256.

On the other hand, the CPL scheme can support very-high-precision arithmetic like $\mathbb{Z}_{2^{2048}}$ and arbitrary modulus through Montgomery reduction, therefore enabling cryptographic applications like RSA2048⁵. Their arithmetic-to-boolean conversion takes only $O(1)$ bootstrapping and is therefore suitable for

⁵ We could support arbitrary plaintext modulus using similar ideas but it is not further explored in this work.

scenarios with very-high-precision arithmetic interleaved with logic operations, under settings when there are a small amount of plaintext messages.

Compared with [Kim25c], we show similar throughput for multiplication while estimating a much better throughput for bitwise operations. The data for bitwise operation in [Kim25c] is estimated using their `ExactReduce` and one discrete CKKS bootstrapping.

Compared with REFHE [BFG⁺25] in 64-bit settings by estimation, our throughput is $15\times$ higher for multiplication and $491\times$ higher for bitwise operations. Compared with TFHE-rs [Zam22], our throughput of 64-bit multiplication is $863\times$ higher while our latency is similar; for 256-bit multiplication our throughput is $2485\times$ higher and our latency is $9\times$ lower; our throughput of bitwise operation is $27\times$ higher.

Recent developments in low-latency CKKS bootstrapping like [CHKS25, CK25, CS25] could replace our CKKS.Sine call for achieving smaller RLWE parameter and this is an interesting future direction. Recent work supports other kinds of arithmetic (e.g. matrix arithmetic [Par25, GL25]) and they are orthogonal to our work.

A *concurrent* and independent work [PZZ25] also supports arithmetic operations on $\mathbb{Z}_{2^{64}}$ and additionally arbitrary plaintext modulus. Their idea is to decompose an integer into digits encrypted in different BFV ciphertexts with redundant representation and execute reduction for managing the multiplication results. However, for bitwise boolean operations and comparison, they will need iterative bootstrapping with the number linear in the bit-width.

9 Conclusion

This work presents a CKKS-style FHE scheme supporting both arithmetic and boolean operations on SIMD slots of machine words, for scenarios with massive amount of data. We crucially amortize the inherently iterative process for conversion between arithmetic and boolean representations to *constant* using our novel triangle encoding and structural properties of the RLWE problem. An interesting open problem is whether it is possible to convert from arithmetic to boolean mode using $O(1)$ bootstrapping without amortization.

Acknowledgments. The authors used ChatGPT 5.2 for grammar and spell checking. We thank Yilei Chen, Yuriy Polyakov, Bingwu Xiang, Jeremy Kun and Yi Zhang for feedback on early drafts of this work. We thank Yangyu Chen for providing machines for the experiments.

References

- AAMP26. Andreea Alexandru, Ahmad Al Badawi, Daniele Micciancio, and Yuriy Polyakov. Application-aware approximate homomorphic encryption: Configuring FHE for practical use. *IACR Communications in Cryptology*, 2(4), 2026.

- ACC⁺18. Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, Satya Lokam, Daniele Micciancio, Dustin Moody, Travis Morrison, Amit Sahai, and Vinod Vaikuntanathan. Homomorphic encryption security standard. Technical report, HomomorphicEncryption.org, Toronto, Canada, November 2018.
- ACG⁺25. Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. Heir: A universal compiler for homomorphic encryption, 2025.
- AKP25. Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 304–337. Springer, Cham, August 2025.
- BAB⁺22. Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R. V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Report 2022/915, 2022.
- BCC⁺22. Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 223–234. ACM Press, November 2022.
- BCC⁺24. Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. Security guidelines for implementing homomorphic encryption. *CiC*, 1(4):26, 2024.
- BCK⁺23. Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, Jai Hyun Park, and Damien Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 37–69. Springer, Cham, August 2023.
- BCKS24. Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with CKKS. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 94–123. Springer, Cham, May 2024.
- BCM⁺25. Jean-Philippe Bossuat, Anamaria Costache, Christian Mouchet, Lea Nürnberg, and Juan Ramón Troncoso-Pastoriza. Accurate and composable noise estimates for CKKS with application to exact HE computation. *IACR Communications in Cryptology*, 2(2), 2025.
- BdBB⁺25. Mathieu Ballandras, Mayeul de Bellabre, Loris Bergerat, Charlotte Bonte, Carl Bootland, Benjamin R. Curtis, Jad Khatib, Jakub Klemsa, Arthur Meyre, Thomas Montagu, Jean-Baptiste Orfila, Nicolas Sarlin, Samuel Tap, and David Testé. *TFHE-rs: A (Practical) Handbook*. Zama, first edition, February 2025.

- BFG⁺25. Zvika Brakerski, Offir Friedman, Daniel Golan, Alon Gurny, Dolev Mutzari, and Ohad Sheinfeld. REFHE: Fully homomorphic ALU. Cryptology ePrint Archive, Report 2025/1449, 2025.
- BGGJ20. Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar P. Jetchev. CHIMERA: Combining Ring-LWE-based Fully Homomorphic Encryption Schemes. *Journal of Mathematical Cryptology*, 14(1):316–338, 2020.
- BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012.
- BJSW25. Olivier Bernard, Marc Joye, Nigel P. Smart, and Michael Walter. Drifting towards better error probabilities in fully homomorphic encryption schemes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 181–211. Springer, Cham, May 2025.
- BK25. Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 338–370. Springer, Cham, August 2025.
- BKS⁺21. Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe DM de Souza, Vinodh Gopal, et al. Intel HEXL (release 1.2). <https://github.com/intel/hexl>, 2021.
- BKSS24. Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 330–360. Springer, Singapore, December 2024.
- BMTH21. Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 587–617. Springer, Cham, October 2021.
- Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012.
- BTH22. Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 521–541. Springer, Cham, June 2022.
- BV11. Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 97–106. IEEE Computer Society Press, October 2011.
- CCH⁺24. Anamaria Costache, Benjamin R. Curtis, Erin Hales, Sean Murphy, Tabitha Ogilvie, and Rachel Player. On the precision loss in approximate homomorphic encryption. In Claude Carlet, Kalikinkar Mandal, and Vincent Rijmen, editors, *SAC 2023*, volume 14201 of *LNCS*, pages 325–345. Springer, Cham, August 2024.
- CCK⁺25. Jung Hee Cheon, Hyeongmin Choe, Minsik Kang, Jaehyung Kim, Seonghak Kim, Johannes Mono, and Taeyeong Noh. Grafting: Decoupled scale

- factors and modulus in RNS-CKKS. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 1098–1112. ACM Press, October 2025.
- CCKS23. Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Damien Stehlé. Homomorphic multiple precision multiplication for CKKS and reduced modulus consumption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 696–710. ACM Press, November 2023.
- CCP⁺24. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPA^D security of exact FHE schemes. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2505–2519. ACM Press, October 2024.
- CCRS26. Marina Checri, Pierre-Emmanuel Clet, Marc Renard, and Renaud Sirdey. Impossibility of CPAD security for a class of FHE schemes. *Cryptology ePrint Archive*, Paper 2026/203, 2026.
- CCS19. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 34–54. Springer, Cham, May 2019.
- CGGI20. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020.
- CHK⁺18. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018.
- CHKS25. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A shallow and highly parallelizable CKKS bootstrapping algorithm. *Cryptology ePrint Archive*, Report 2025/784, 2025.
- CHM⁺25. Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. MatriGear: Accelerating authenticated matrix triple generation with scalable prime fields via optimized HE packing. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 2453–2471. IEEE Computer Society Press, May 2025.
- CK25. Jean-Sébastien Coron and Robin Köstler. Low-latency bootstrapping for CKKS using roots of unity. *Cryptology ePrint Archive*, Report 2025/651, 2025.
- CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017.
- CLPX18. Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In Nigel P. Smart, editor, *CT-RSA 2018*, volume 10808 of *LNCS*, pages 116–136. Springer, Cham, April 2018.
- CPL25. Gyeongwon Cha, Dongjin Park, and Joon-Woo Lee. Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. *Cryptology ePrint Archive*, Report 2025/1740, 2025.

- CS25. Jean-Sébastien Coron and Tim Seuré. PaCo: Bootstrapping for CKKS via partial CoeffToSlot. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VII*, volume 16251 of *LNCS*, pages 36–67. Springer, Singapore, December 2025.
- CSBB24. Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPA^D security of “exact” and threshold FHE schemes and libraries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 3–33. Springer, Cham, August 2024.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015.
- DMPS24. Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *Journal of Cryptology*, 37(1):3, January 2024.
- EGR25. Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS ’25, page 734–749, New York, NY, USA, 2025. Association for Computing Machinery.
- FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.
- Gen09. Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, 2009. crypto.stanford.edu/craig.
- GHS12. Craig Gentry, Shai Halevi, and Nigel P. Smart. Homomorphic evaluation of the AES circuit. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 850–867. Springer, Berlin, Heidelberg, August 2012.
- GL25. Craig Gentry and Yongwoo Lee. Fully homomorphic encryption for matrix arithmetic. Cryptology ePrint Archive, Report 2025/1935, 2025.
- GNSJ24. Qian Guo, Denis Nabokov, Elias Suvanto, and Thomas Johansson. Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
- GSW13. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 75–92. Springer, Berlin, Heidelberg, August 2013.
- GV25. Robin Geelen and Frederik Vercauteren. Fully homomorphic encryption for cyclotomic prime moduli. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 366–397. Springer, Cham, May 2025.
- GZ25. Mingyu Gao and Hongren Zheng. A critique on average-case noise analysis in rlwe-based homomorphic encryption. In *Proceedings of the 13th Workshop on Encrypted Computing & Applied Homomorphic Computing*, WAHC ’25, page 26–37, New York, NY, USA, 2025. Association for Computing Machinery.

- HS14. Shai Halevi and Victor Shoup. Algorithms in HElib. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 554–571. Springer, Berlin, Heidelberg, August 2014.
- JP⁺24. Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. NeuJeans: Private neural network inference with joint optimization of convolution and FHE bootstrapping. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 4361–4375. ACM Press, October 2024.
- KDE⁺24. Andrey Kim, Maxim Deryabin, Jieun Eom, Rakyong Choi, Yongwoo Lee, Whan Ghang, and Donghoon Yoo. General bootstrapping approach for rlwe-based homomorphic encryption. *IEEE Transactions on Computers*, 73(1):86–96, 2024.
- Kim25a. Jaehyung Kim. Bootstrapping GBFV with CKKS. Cryptology ePrint Archive, Report 2025/888, 2025.
- Kim25b. Jaehyung Kim. Efficient homomorphic integer computer from CKKS. Cryptology ePrint Archive, Report 2025/066, 2025.
- Kim25c. Jaehyung Kim. Faster homomorphic integer computer. Cryptology ePrint Archive, Report 2025/1440, 2025.
- KN25. Jaehyung Kim and Taeyeon Noh. Modular reduction in CKKS. *IACR Communications in Cryptology*, 2(2), 2025.
- KPP22. Andrey Kim, Antonis Papadimitriou, and Yuriy Polyakov. Approximate homomorphic encryption with reduced approximation error. In Steven D. Galbraith, editor, *CT-RSA 2022*, volume 13161 of *LNCS*, pages 120–144. Springer, Cham, March 2022.
- KPZ21. Andrey Kim, Yuriy Polyakov, and Vincent Zucca. Revisiting homomorphic encryption schemes for finite fields. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 608–639. Springer, Cham, December 2021.
- KSS24. Jaehyung Kim, Jinyeong Seo, and Yongsoo Song. Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2535–2546. ACM Press, October 2024.
- LHH⁺21. Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. Pegasus: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073. IEEE, 2021.
- LLL⁺21. Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 618–647. Springer, Cham, October 2021.
- LM21. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 648–677. Springer, Cham, October 2021.
- LMK⁺23. Yongwoo Lee, Daniele Micciancio, Andrey Kim, Rakyong Choi, Maxim Deryabin, Jieun Eom, and Donghoon Yoo. Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In Carmit Hazay and Martijn Stam, editors, *EURO-*

- CRYPTO 2023, Part III*, volume 14006 of *LNCS*, pages 227–256. Springer, Cham, April 2023.
- LMP22. Zeyu Liu, Daniele Micciancio, and Yuriy Polyakov. Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 130–160. Springer, Cham, December 2022.
- LMSS22. Baiyu Li, Daniele Micciancio, Mark Schultz, and Jessica Sorrell. Securing approximate homomorphic encryption using differential privacy. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 560–589. Springer, Cham, August 2022.
- LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010.
- LPR13. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-LWE cryptography. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013.
- MML⁺23. Johannes Mono, Chiara Marcolla, Georg Land, Tim Güneysu, and Najwa Aaraj. Finding and evaluating parameters for BGV. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 370–394. Springer, Cham, July 2023.
- Par25. Jai Hyun Park. Ciphertext-ciphertext matrix multiplication: Fast for large matrices. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 153–180. Springer, Cham, May 2025.
- PZZ25. Chris Peikert, Doron Zarchy, and Guy Zyskind. High-precision exact FHE made simple, general, and fast. Cryptology ePrint Archive, Paper 2025/2321, 2025.
- RAD78. Ronald L Rivest, Len Adleman, and Michael L Dertouzos. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.
- Reg05. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.
- SSTX09. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 617–635. Springer, Berlin, Heidelberg, December 2009.
- SV10. Nigel P. Smart and Frederik Vercauteren. Fully homomorphic encryption with relatively small key and ciphertext sizes. In Phong Q. Nguyen and David Pointcheval, editors, *PKC 2010*, volume 6056 of *LNCS*, pages 420–443. Springer, Berlin, Heidelberg, May 2010.
- vDGHV10. Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. Fully homomorphic encryption over the integers. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 24–43. Springer, Berlin, Heidelberg, May / June 2010.
- Zam22. Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. <https://github.com/zama-ai/tfhe-rs>.

A Additional Preliminaries

A.1 Rings and their relations

Isomorphisms in matrix form. In the RLWE ring $\mathcal{R}_{\mathbb{R}}$, fix a polynomial $m(X)$ and identify it as a vector $\mathbf{m} = (m_0, m_1, \dots, m_{N-1}) \in \mathbb{R}^N$, σ can be understood as a $(N/2) \times N$ Vandermonde matrix $\mathbf{U}_{\sigma}$ such that $\mathbf{z} = \mathbf{U}_{\sigma} \mathbf{m}$ with $\mathbf{z} \in \mathbb{C}^{N/2}$.

$$\mathbf{U}_{\sigma} = \begin{bmatrix} 1 & \zeta_0 & \zeta_0^2 & \cdots & \zeta_0^{N-1} \\ 1 & \zeta_1 & \zeta_1^2 & \cdots & \zeta_1^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{N/2-1} & \zeta_{N/2-1}^2 & \cdots & \zeta_{N/2-1}^{N-1} \end{bmatrix}$$

Consider the $N \times N$ matrix $\text{CRT}_{\sigma} = (\mathbf{U}_{\sigma}; \overline{\mathbf{U}_{\sigma}})$, its inverse is $\text{CRT}_{\sigma}^{-1} = \frac{1}{N} \overline{\text{CRT}_{\sigma}}^T$. Since we have $(\mathbf{z}; \overline{\mathbf{z}}) = \text{CRT}_{\sigma} \cdot \mathbf{m}$, the inverse transform can be evaluated as

$$\mathbf{m} = \frac{1}{N} \left(\overline{\mathbf{U}_{\sigma}}^T \cdot \mathbf{z} + \mathbf{U}_{\sigma}^T \cdot \overline{\mathbf{z}} \right)$$

We can further divide $\mathbf{U}_{\sigma}$ into two $(N/2) \times (N/2)$ square matrices $\mathbf{U}_{\sigma} = [\mathbf{U}_{\sigma,0} \quad \mathbf{U}_{\sigma,1}]$. Note that $\mathbf{U}_{\sigma,1} = \sqrt{-1} \cdot \mathbf{U}_{\sigma,0}$. We use the notation $\mathbf{V}_{\sigma,0} = \overline{\mathbf{U}_{\sigma,0}}^T$ and $\mathbf{V}_{\sigma,1} = \overline{\mathbf{U}_{\sigma,1}}^T$. Let $\mathbf{m} = (\mathbf{m}_0; \mathbf{m}_1)$ where $\mathbf{m}_0 = (m_0, m_1, \dots, m_{N/2-1}) \in \mathbb{R}^{N/2}$ and $\mathbf{m}_1 = (m_{N/2}, \dots, m_{N-1}) \in \mathbb{R}^{N/2}$, we have

$$\begin{aligned} \mathbf{m}_0 &= \frac{2}{N} \Re[\mathbf{V}_{\sigma,0} \cdot \mathbf{z}] = \frac{1}{N} (\mathbf{V}_{\sigma,0} \cdot \mathbf{z} + \overline{\mathbf{V}_{\sigma,0} \cdot \mathbf{z}}) \\ \mathbf{m}_1 &= \frac{2}{N} \Im[\mathbf{V}_{\sigma,0} \cdot \mathbf{z}] = \frac{-\sqrt{-1}}{N} (\mathbf{V}_{\sigma,0} \cdot \mathbf{z} - \overline{\mathbf{V}_{\sigma,0} \cdot \mathbf{z}}) \\ \mathbf{z} &= \mathbf{U}_{\sigma,0}(\mathbf{m}_0 + \sqrt{-1} \cdot \mathbf{m}_1) \end{aligned}$$

In the arithmetic ring $\mathcal{Z}_{\mathbb{R}}$, fix a polynomial $m(X)$ and identify it as a vector $\mathbf{m} = (m_0, m_1, \dots, m_{n-1})$. τ can be understood as $(n/2) \times n$ Vandermonde matrix $\mathbf{U}_{\tau}$ such that $\mathbf{z} = \mathbf{U}_{\tau} \mathbf{m}$ with $\mathbf{z} \in \mathbb{C}^{n/2}$.

$$\mathbf{U}_{\tau} = \begin{bmatrix} 1 & \xi_0 & \xi_0^2 & \cdots & \xi_0^{n-1} \\ 1 & \xi_1 & \xi_1^2 & \cdots & \xi_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \xi_{n/2-1} & \xi_{n/2-1}^2 & \cdots & \xi_{n/2-1}^{n-1} \end{bmatrix}$$

For $\text{CRT}_{\tau} = (\mathbf{U}_{\tau}; \overline{\mathbf{U}_{\tau}})$, we denote $\tau^{-1} = \text{CRT}_{\tau}^{-1}$. Vandermonde matrix theory tells us that the k -th column of the inverse τ^{-1} has the coefficients of the k -th Lagrange polynomial

$$\ell_k(x) = \frac{1}{n\xi_k^{n-1} - 1} \left(-1 + \xi_k^{n-1} + \sum_{i=1}^{n-1} \xi_k^{n-i-1} X^i \right)$$

Let $\ell_k(X) = \sum_{i=0}^{n-1} \ell_{k,i} \cdot X^i$, τ^{-1} can be expressed as $\tau^{-1} = [\mathbf{V}_\tau \quad \overline{\mathbf{V}_\tau}]$ where $\mathbf{V}_\tau$ is a $n \times (n/2)$ matrix

$$\mathbf{V}_\tau = \begin{bmatrix} \ell_{0,0} & \ell_{1,0} & \cdots & \ell_{n/2-1,0} \\ \ell_{0,1} & \ell_{1,1} & \cdots & \ell_{n/2-1,1} \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{0,n-1} & \ell_{1,n-1} & \cdots & \ell_{n/2-1,n-1} \end{bmatrix}$$

This gives us the following relation between $\mathbf{z}$ and $\mathbf{m}$.

$$\mathbf{m} = \mathbf{V}_\tau \cdot \mathbf{z} + \overline{\mathbf{V}_\tau \cdot \mathbf{z}} \quad \mathbf{z} = \mathbf{U}_\tau \cdot \mathbf{m}$$

If we divide them into $(n/2) \times (n/2)$ square matrices $\mathbf{U}_\tau = [\mathbf{U}_{\tau,0} \quad \mathbf{U}_{\tau,1}]$ and $\mathbf{V}_\tau = (\mathbf{V}_{\tau,0}; \mathbf{V}_{\tau,1})$, we have

$$\begin{aligned} \mathbf{m}_0 &= 2\Re[\mathbf{V}_{\tau,0} \cdot \mathbf{z}] & \mathbf{m}_1 &= 2\Re[\mathbf{V}_{\tau,1} \cdot \mathbf{z}] \\ \mathbf{z} &= \mathbf{U}_{\tau,0} \cdot \mathbf{m}_0 + \mathbf{U}_{\tau,1} \cdot \mathbf{m}_1 \end{aligned}$$

Automorphisms. We did not find interesting applications of automorphism for the arithmetic ring $\mathcal{Z}_{\mathbb{R}}$. We detail the discussion on the RLWE ring $\mathcal{R}$.

Let one possible automorphism index set be $\langle 5, -1 \rangle$ for the multiplicative group $\mathbb{Z}_{2N}^*$, let j be an automorphism index, the j -th automorphism is

$$\begin{aligned} \phi_j : \mathcal{R} &\rightarrow \mathcal{R} \\ m(X) &\mapsto m(X^j) \end{aligned}$$

It is noted that

$$\begin{aligned} \sigma(\phi_{5^i}(m(X))) &= (\dots, m(\zeta_k^{5^i}), \dots) = (\dots, m(\zeta_{k+i \pmod{N/2}}), \dots) \\ \sigma(\phi_{-1}(m(X))) &= (\dots, m(\zeta_k^{-1}), \dots) = (\dots, \overline{m(\zeta_k)}, \dots) \end{aligned}$$

which means automorphism indices of form 5^i gives rotation while -1 gives conjugation in the canonical embedding.

Subring and Sparse Packing. We mainly discuss the subring structure for $\mathcal{R} = \mathbb{Z}[X]/\langle X^N + 1 \rangle$. It is noted that the ring $\mathcal{R}' = \mathbb{Z}[Y]/\langle Y^\ell + 1 \rangle$ can be embedded into $\mathcal{R}$ using $Y = X^{N/\ell}$ when ℓ is a divisor of N . For $\mathcal{R}'$, one primitive root is $\zeta' = \exp(2\pi\sqrt{-1}/(2\ell)) = \zeta^{N/\ell}$. We can again define a $\mathcal{R}'$ -canonical embedding σ' and identify an element in $\mathcal{R}'$ with a vector in $\mathbb{C}^{\ell/2}$ using $\zeta'_j = \zeta'^{5^j}$ for $0 \leq j < \ell/2$. Now for a $m(Y) \in \mathcal{R}'$ with $\sigma'(m) = \mathbf{w} = (w_j)_{0 \leq j < \ell/2}$, its embedding $\tilde{m}(X) = m(X^{N/\ell})$ in $\mathcal{R}$ has the canonical embedding $\mathbf{z} = (z_j)_{0 \leq j < N/2} = \sigma(\tilde{m})$ as

$$z_j = \tilde{m}(\zeta_j) = m(\zeta_j^{N/\ell}) = m(\zeta'_j \pmod{\ell/2}) = w_j \pmod{\ell/2}, \quad 0 \leq j < N/2$$

so $\mathbf{z}$ can be understood as a concatenation of N/ℓ copies of $\mathbf{w}$.

Conversely, given a $\tilde{m}(X)$ in $\mathcal{R}$, Galois theory tells us that if we sum over all automorphisms that fixes $\mathcal{R}' \subset \mathcal{R}$, we obtain a $m(X^{N/\ell})$ that can be interpreted

as $m(Y) \subset \mathcal{R}'$. This is the field trace function and by legacy we call it the **PartialSum** function [CHK⁺18]. Such process can also be viewed as turning the canonical embedding $\mathbf{z}$ of $\tilde{m}(X)$ into a concatenation of N/ℓ copies of $\mathbf{w}$ for some $m(Y)$ using $\log(N/\ell)$ rotations.

The relevance of subring structure comes from the fact that in certain circumstances we may only want to connect $\mathcal{R}'$ instead of $\mathcal{R}$ with ℓ/n copies of $\mathcal{Z}_{\mathbb{R}}$, commonly known as *sparse packing*. The linear transformation now, as the $\mathcal{R}' \subset \mathcal{R}$ embedding naturally gives repetition, can be written down as

$$\mathbf{m} = \begin{bmatrix} \mathbf{m}_0 \\ \mathbf{m}_1 \end{bmatrix} = \frac{2}{\ell} \Re \left(\begin{bmatrix} \mathbf{V}_{\sigma,0} & \\ & \mathbf{V}_{\sigma,1} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{z} \\ \mathbf{z} \end{bmatrix} \right) \quad \begin{bmatrix} \mathbf{z}_0 \\ \mathbf{z}_1 \end{bmatrix} = \begin{bmatrix} \mathbf{U}_{\sigma,0} & \\ & \mathbf{U}_{\sigma,1} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{m}_0 \\ \mathbf{m}_1 \end{bmatrix}$$

Further rotation gives $(\mathbf{z}_0; \mathbf{z}_1) + (\mathbf{z}_1; \mathbf{z}_0) = (\mathbf{z}; \mathbf{z})$ and can be viewed as canonical embedding of subring elements. Similar linear transformation for τ can be defined here if we replace every subscript σ with τ (and remove the $\frac{1}{\ell}$ coefficient).

For ease of discussion, we assume *full packing* (i.e., $\ell = N$) throughout this work.

A.2 RLWE operations

We discuss several interesting *modulo* operations we can do on the ciphertext. They are the cornerstone of various management and bootstrapping operations.

Definition 19 (RLWE Rescaling). *Given $\mathbf{ct} = (b, a) = (-a \cdot s + m + e, a) \in \mathcal{R}_{Q_L}^2$. Suppose $Q_L = Q_{L-1}q_L$, we can rescale from $\mathcal{R}_{Q_L}$ to $\mathcal{R}_{Q_{L-1}}$ by the following operation*

$$\mathbf{ct}' = \text{RLWE.Rescale}_{q_L}(\mathbf{ct}) = \frac{\mathbf{ct} + \delta}{q_L} \\ \text{with } \delta = ([-b]_{q_L}, [-a]_{q_L})$$

We note the effect on the message as follows. In $\mathcal{R}$, we have $\langle \delta, (1, s) \rangle = -[m + e]_{q_L} + q_L I$, so it follows that

$$\text{RLWE.Decrypt}_s(\mathbf{ct}') = \left\lfloor \frac{m + e}{q_L} \right\rfloor + I \pmod{Q_{L-1}}$$

We call I the rescaling noise and it follows the same distribution as in Prop 1 with hamming weight $h = 192$.

We now discuss the arithmetic of the RLWE ring, and its effect on the message m .

Definition 20 (RLWE Addition and Tensor Product). *Given $\mathbf{ct} = (b, a) = (-as + m + e, a) \in \mathcal{R}_Q^2$ and $\mathbf{ct}' = (b', a') = (-a's + m + e', a') \in \mathcal{R}_Q^2$, the addition and tensor product operations are*

$$\begin{aligned} \text{RLWE.Add}(\mathbf{ct}, \mathbf{ct}') &= (b + b', a + a') \in \mathcal{R}_Q^2 \\ \text{RLWE.Tensor}(\mathbf{ct}, \mathbf{ct}') &= (b \cdot b', a \cdot b' + a' \cdot b, a \cdot a') \in \mathcal{R}_Q^3 \end{aligned}$$

where addition gives $m + m'$ and tensor product gives $m \cdot m'$. $\text{RLWE.Tensor}(\mathbf{ct}, \mathbf{ct}')$ should be decrypted using $(1, s, s^2)$.

Definition 21 (RLWE Scalar Multiplication). Given $\text{ct} = (b, a) = (-a \cdot s + m + e, a) \in \mathcal{R}_Q^2$, for a scalar $k \in \mathcal{R}_Q$ or $k \in \mathbb{Z}_Q$, we define the scalar multiplication operation

$$\text{ct}' = \text{RLWE.MultScalar}(\text{ct}, k) = (k \cdot b, k \cdot a) \in \mathcal{R}_Q^2$$

The effect on the message is

$$\text{RLWE.Decrypt}_s(\text{ct}') \equiv [km + ke]_{Q_L} \pmod{Q}$$

We now discuss how automorphisms of $\mathcal{R}$ translates into automorphisms in RLWE encrypted ciphertext.

Definition 22 (RLWE Automorphism). Given $\text{ct} = (b, a) = (-as + m + e, a) \in \mathcal{R}_Q^2$, since $\mathcal{R}_Q = \mathbb{Z}[X]/\langle X^N + 1, Q \rangle$, we can also recognize elements as polynomials like $b(X), a(X), m(X)$. Given the automorphism index set $\langle 5, -1 \rangle \subset \mathbb{Z}_{2N}^*$. Let j be an automorphism index, we define the j -th automorphism as

$$\begin{aligned} \text{ct}' &= \text{RLWE.Automorphism}_j(\text{ct}) = (b(X^j), a(X^j)) \\ &= (-a(X^j)s(X^j) + m(X^j) + e(X^j), a(X^j)) \end{aligned}$$

we note that ct' is an encryption of $m(X^j)$ using $s(X^j)$.

The RLWE tensor product and automorphism operation will change the decryption basis, which will propagate along the computation. To ease the management, the key switching technique was proposed [BV11]. It is observed that for an element $d' \in \mathcal{R}_Q$ that should be decrypted using s' , we can *homomorphically* decrypt it using $\text{RLWE.Encrypt}_s(s')$ like

$$\begin{aligned} d &= d' \cdot \text{RLWE.Encrypt}_s(s') = \text{RLWE.Encrypt}_s(d's') \\ &= (-as + d's' + d'e, a) \end{aligned}$$

so that d can be now decrypted using s . On the other hand, the direct production of $d \in \mathcal{R}_Q$ may cause a large $d'e$ noise term, so gadget decomposition techniques were proposed to reduce the growth of the noise.

Definition 23 (RLWE Key Switching). Given $\text{ct}_{s'} = (b, a) = (-as' + m + e, a)$ that should be decrypted using $(1, s')$, the key switching operation is

$$\text{ct}_s = \text{RLWE.KeySwitch}_{s' \rightarrow s}(\text{ct}_{s'}) = (b, 0) + \mathcal{D}(a) \cdot \text{RLWE.Encrypt}_s(\mathcal{P}(s'))$$

where $\mathcal{D}(\cdot)$ and $\mathcal{P}(\cdot)$ is a pair of decomposition and power-up function such that $\mathcal{D}(x) \cdot \mathcal{P}(y) = x \cdot y \in \mathcal{R}_Q$. There exists a pair of $\mathcal{D}$ and $\mathcal{P}$ such that the additional noise caused is nearly the same as the RLWE rescaling operation.

The $\text{RLWE.Encrypt}_s(\mathcal{P}(s'))$ is the evaluation key.

Typical construction and analysis of $\mathcal{D}$ and $\mathcal{P}$ can be found in [KPZ21, MML⁺23]. It is further noted that the evaluation key is an encryption of the secret key itself, thus requiring an additional assumption called circular secure assumption.

A.3 CKKS operations

Definition 24 (CKKS Rotation and Conjugation). *Given a polynomial $m(X) \in \mathcal{R}_Q$ that encodes $\mathbf{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{C}^{N/2}$, for rotation index i , we define the automorphism index to be $j = 5^i \pmod{2N}$, then we define the rotation and conjugation operation to be*

$$\begin{aligned} m'(X) &= \text{CKKS.Rotate}_i(m(X)) = m(X^j) \\ m''(X) &= \text{CKKS.Conjugate}(m(X)) = m(X^{-1}) \end{aligned}$$

where $m(X^j)$ can be homomorphically evaluated using $\text{RLWE.Automorphism}_j$. Note that the k -th slot of $m'(X)$ is now the $(k+i)$ -th (modulo $N/2$) slot of $m(X)$, and the k -th slot of $m''(X)$ is the conjugate of the k -th slot of $m(X)$.

Definition 25 (CKKS Sine Operation). *For a CKKS encoded data $qI + \Delta z + e \in \mathbb{C}$ where $\Delta z + e \lll q$, $I \in \mathbb{Z}$ and $|I| \leq K$, let $\epsilon = \frac{\Delta z + e}{q}$, there exists a polynomial that evaluates*

$$\begin{aligned} \text{CKKS.Sine}(qI + \Delta z + e) &= \frac{q}{2\pi} \sin\left(\frac{2\pi}{q}(qI + \Delta z + e)\right) \approx q(\epsilon + O(|\epsilon|^3)) + e' \\ &= \Delta z + e + qO(|\epsilon|^3) + e' \end{aligned}$$

where $O(|\epsilon|^3)$ is the error from approximating x using $\sin(x)$, and e' consists of noises like polynomial approximation error and multiplication noises.

Using automorphisms in $\mathcal{R}$, we can construct operations like CKKS.Rotate and CKKS.Conjugate (Def. 24). With them, we can efficiently evaluate linear transformations. We present a simple Halevi-Shoup style linear transformation [HS14] and there are many optimizations available like using Fast Fourier Transform [CCS19].

Definition 26 (CKKS Linear Transformation). *For a matrix $\mathbf{A} \in \mathbb{C}^{N/2 \times N/2}$, given a polynomial $m(X) \in \mathcal{R}_Q$ that encodes $\mathbf{z} = (z_0, z_1, \dots, z_{N/2-1}) \in \mathbb{C}^{N/2}$, there exists an algorithm to evaluate the matrix multiplication*

$$\begin{aligned} \text{CKKS.Ecd}_{\Delta, \mathcal{R}_Q}(\mathbf{A} \cdot \mathbf{z}) &= \text{CKKS.LT}_{\mathbf{A}}(m(X)) \\ &= \sum_{0 \leq i < N/2} u_i(X) \cdot \text{CKKS.Rotate}_i(m(X)) \end{aligned}$$

$$\text{with } u_i(X) = \text{CKKS.Ecd}_{\Delta, \mathcal{R}_Q}(\text{diag}_i(\mathbf{A}))$$

where $\text{diag}_i(\mathbf{A})$ is the i -th diagonal $(A_{0,i}, A_{1,i+1}, \dots)$ of $\mathbf{A}$.

B Boolean Flattened Mode

Now assume we are able to extract bits from $\mathcal{Z}_{\mathbb{R}}$ encoded polynomial to $\mathbb{C}^n$ via bootstrapping. This is exactly discrete CKKS with plaintext modulus of 2. For illustration purpose, we define some example boolean operations as follows.

Definition 27 (Boolean Encoding in $\mathbb{C}$). For a message $m \in \mathbb{Z}_{2^n}$ such that $m = \sum_{i=0}^{n-1} b_i 2^i$, let $\mathbf{b} = (b_0, b_1, \dots, b_{n-1})$, its encoding in $\mathbb{C}^n$ is

$$\mathbb{C}.\text{Ecd}_\Delta(m) = \Delta \mathbf{b}$$

Definition 28 (Boolean Bitwise Operations in $\mathbb{C}$). For two messages $\Delta \mathbf{b} = \mathbb{C}.\text{Ecd}_\Delta(m)$ and $\Delta \mathbf{b}' = \mathbb{C}.\text{Ecd}_\Delta(m')$, their boolean operation can be achieved by

$$\begin{aligned} \text{AND} &= \frac{1}{\Delta} \cdot \Delta \mathbf{b} \cdot \Delta \mathbf{b}' \\ \text{OR} &= \Delta \mathbf{b} + \Delta \mathbf{b}' - \frac{1}{\Delta} \cdot \Delta \mathbf{b} \cdot \Delta \mathbf{b}' \\ \text{XOR} &= \Delta \mathbf{b} + \Delta \mathbf{b}' - \frac{2}{\Delta} \cdot \Delta \mathbf{b} \cdot \Delta \mathbf{b}' \\ \text{NOT} &= \Delta \cdot 1 - \Delta \mathbf{b} \end{aligned}$$

Other bitwise boolean operations like NAND can be achieved similarly.

The ability of CKKS.Rotate to rotate the $\mathbb{C}$ -slots naturally gives us the ability to shift and rotate the bits.

Definition 29 (Bit Rotation/Shifting Operations in $\mathbb{C}$). For a vector $\Delta \mathbf{b} = \Delta(b_0, b_1, \dots, b_{n-1}) \in \mathbb{C}^n$, if we are given rotation operation and arithmetic operations for $\mathbb{C}^n$, we can achieve bit rotation and bit shifting via masking, rotation and recombination.

The masking and recombination can be implemented using AND and OR.

Definition 30 (Sign extraction in $\mathbb{C}$). Given an encoding of m in $\mathbb{C}^n$ as $\mathbb{C}.\text{Ecd}_\Delta(m) = \Delta \mathbf{b} = \Delta(b_0, b_1, \dots, b_{n-1})$, the extraction of the sign of m is the extraction of b_{n-1} and can be done through masking and bit shifting.

With sign extraction, further operations like comparison can be implemented using subtraction in arithmetic mode and sign extraction in boolean mode.

C Additional Illustration for Arithmetic Triangle Encoding

If we multiply the arithmetic triangle encoding (Fig. 1) by $t = X - 2$, we obtained a binary representation but the bits are mixed with tI which is hard to separate without consulting iterative process.

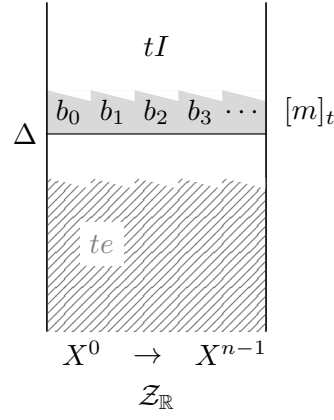


Fig. 10. Arithmetic triangle encoding after multiplication by t .